



Performance Analysis



Reattempt



Download Certificate

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.



Answer Distribution

0

Correct

0

Incorrect

65

Unattempted



Domain Performance

Compute	0%
Security	0%
Serverless	0%
Networking	0%
Disaster Recovery	0%
Database	0%
Monitoring and Logging	0%
Storage	0%

Final Score

**0/65**

Passing score: 48

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis

Correct	0
Incorrect	0
Unattempted	65
Reviewed	0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Compute



A company is migrating its on-premises containerized application to a hybrid environment using Amazon Elastic Container Service (Amazon ECS). The application must run both on-premises and in the AWS Cloud, utilizing containers for consistency and scalability. The company requires a solution that allows seamless scaling across both environments, with new application containers running in the AWS Cloud and utilizing a load balancer to manage HTTP traffic. Which combination of actions will satisfy these requirements?

- ✔ Set up an ECS cluster that uses the AWS Fargate launch type for the cloud application containers. Use an Amazon ECS Anywhere external launch type for the on-premises application containers.

This option allows you to run containers in the cloud using Fargate (serverless) and extend ECS to your on-premises environment with ECS Anywhere. ECS Anywhere enables you to register your on-premises servers or VMs as part of your ECS cluster.

- ✔ Set up an Application Load Balancer for cloud ECS services.

Using an Application Load Balancer is essential for distributing HTTP traffic across the ECS containers running in the cloud. It provides the necessary load balancing and traffic management capabilities for web-based applications.

- ✘ Set up a Network Load Balancer for cloud ECS services.

A Network Load Balancer is more suited for TCP or UDP traffic, not HTTP traffic as specified in the requirements. Therefore, this option is not optimal.

- ✘ Set up an ECS cluster that uses the AWS Fargate launch type. Use Fargate for the cloud application containers and the on-premises application containers.

Fargate cannot be directly used for on-premises environments without ECS Anywhere. Therefore, this option is not valid for the hybrid setup.

- ✘ Set up an ECS cluster that uses the Amazon EC2 launch type for the cloud application containers. Use Amazon ECS Anywhere with an AWS Fargate launch type for the on-premises application containers.

Using EC2 launch type for the cloud environment is a valid option but it requires more operational overhead compared to Fargate. Also, the Fargate launch type cannot be directly used with ECS Anywhere on-premises. ECS Anywhere uses external instances registered to the ECS control plane.

Overall Explanation:

The correct combination includes setting up an ECS cluster with Fargate for the cloud and ECS Anywhere for on-premises to provide a scalable, hybrid, and cloud-native solution. An Application Load Balancer is needed to balance HTTP traffic across the containers running in the cloud. The combination of using Fargate and ECS Anywhere allows the customer to run the same containerized application in both environments, reducing operational overhead and ensuring consistent behavior.

 View Discussions

 Ask AI

Question 2

Not Attempted

Security



A company is migrating its on-premises workloads, including sensitive SQL Server databases, to AWS. The company wants to enhance database security and minimize operational overhead. Which solution best meets these requirements?

- ☐ Migrate the databases to Amazon EC2 instances. Use an AWS Key Management Service (AWS KMS) AWS managed key for encryption.

While EC2 provides flexibility, it increases operational overhead as you're responsible for managing the OS, database software, patching, and backups.

- ☒ Migrate the databases to a Multi-AZ Amazon RDS for SQL Server DB instance. Use an AWS Key Management Service (AWS KMS) AWS managed key for encryption.

This option provides a managed database service (RDS) that reduces operational overhead and offers enhanced security features like encryption at rest using KMS and high availability with Multi-AZ deployment.

- ☐ Migrate the data to an Amazon S3 bucket. Use Amazon Macie to ensure data security.

S3 is object storage, not a relational database. Migrating data to S3 would require significant application changes and wouldn't meet the requirement of using a SQL Server database.

- ☐ Migrate the databases to an Amazon DynamoDB table. Use Amazon CloudWatch Logs to ensure data security.

DynamoDB is a NoSQL database, not a relational database like SQL Server. This would require significant application changes. CloudWatch Logs is for logging and monitoring, not data security itself.

Overall Explanation:

Using a Multi-AZ Amazon RDS for SQL Server DB instance with KMS encryption offers the best balance between increased security (through encryption and high availability) and reduced operational overhead (through managed database services) while maintaining the existing relational database structure. EC2 requires more management overhead. S3 and DynamoDB are not suitable replacements for SQL Server without significant application changes.

[View Discussions](#)[Ask AI](#)

Question 3

Not Attempted

Security



A company wants to migrate an application to AWS and improve its availability. They also want to implement AWS WAF to protect the application. Which solution best meets these requirements?

- ☒ Create an Auto Scaling group containing multiple Amazon EC2 instances hosting the application across two Availability Zones. Configure an Application Load Balancer (ALB) and set the Auto Scaling group as the target. Connect AWS WAF to the ALB.

This solution provides high availability through the Auto Scaling group and multiple Availability Zones. AWS WAF protects the application by filtering malicious traffic at the ALB level.

- ☐ Create a cluster placement group containing multiple Amazon EC2 instances that host the application. Configure an Application Load Balancer and set the EC2 instances as the targets. Connect a WAF to the placement group.

Cluster placement groups offer low latency but do not inherently provide high availability across Availability Zones. WAF cannot be directly connected to placement group.

- ☐ Create two Amazon EC2 instances hosting the application across two Availability Zones. Configure the EC2 instances as the targets of an Application Load Balancer (ALB). Connect a WAF to the ALB.

While this solution uses multiple Availability Zones, it does not scale automatically and therefore doesn't maximize availability as much as an Auto Scaling group.

- ☐ Create an Auto Scaling group containing multiple Amazon EC2 instances hosting the application across two Availability Zones. Configure an Application Load Balancer (ALB) and set the Auto Scaling group as the target. Connect a WAF to the Auto Scaling group.

AWS WAF cannot be directly associated with an Auto Scaling Group. It must be associated with a supported resource such as an Application Load Balancer.

Overall Explanation:

The best solution is to use an Auto Scaling group across multiple Availability Zones for high availability and an Application Load Balancer (ALB) with AWS WAF attached for application protection. The WAF filters traffic before it reaches the EC2 instances in the Auto Scaling Group. The flow is: WAF -> ALB -> Auto Scaling Group (EC2 instances).

[View Discussions](#)[Ask AI](#)

Question 4

Not Attempted

Security



A company manages a data lake in an Amazon S3 bucket that numerous applications access. The S3 bucket contains a unique prefix for each application. The company wants to restrict each application to its specific prefix and to have granular control of the objects under each prefix. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ Create dedicated S3 access points and access point policies for each application.

By creating separate access points for each application, you can enforce access controls specific to their respective prefixes while minimizing administrative complexity. This approach provides a clean separation of permissions and reduces the risk of misconfigurations. S3 Access Points provide a way to manage access to shared data sets in Amazon S3. Each access point has a unique hostname and a policy that is specific to the use case, allowing for granular control over access to data. Access Point Policies can be tailored to restrict access to specific prefixes within an S3 bucket, ensuring that each application only has access to its designated prefix.

- ☐ Create an S3 Batch Operations job to set the ACL permissions for each object in the S3 bucket.

S3 Batch Operations involves setting ACL permissions for each object individually, which can be cumbersome and time-consuming. Also, ACL is not recommended by AWS.

- ☐ Replicate the objects in the S3 bucket to new S3 buckets for each application. Create replication rules by prefix.

Replicating objects to new S3 buckets introduces additional buckets and replication rules, increasing management overhead.

- ☐ Replicate the objects in the S3 bucket to new S3 buckets for each application. Create dedicated S3 access points for each application.

This option adds unnecessary complexity by combining replication and access point creation.

Overall Explanation:

The most efficient solution is to create dedicated S3 access points and access point policies for each application. This approach provides granular control and restricts each application to its specific prefix with minimal operational overhead. Options B, C, and D are less efficient due to increased complexity and management overhead.

[View Discussions](#)[Ask AI](#)

Question 5

Not Attempted

Serverless



A company's application allows customers to upload images to an Amazon S3 bucket. Currently, a nightly Amazon EC2 Spot Fleet processes all images received that day. Each image takes 2 minutes to process and requires 512 MB of memory. A solutions architect needs to modify the application to process images immediately upon upload. Which change will meet these requirements MOST cost-effectively?

- ☒ Use S3 Event Notifications to write a message with image details to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an AWS Lambda function to read the messages from the queue and to process the images.

This is the most cost-effective solution. Lambda is well-suited for processing tasks under 15 minutes with sufficient memory (up to 10 GB). Using SQS ensures that all images are processed reliably.

- ☐ Use S3 Event Notifications to write a message with image details to an Amazon Simple Queue Service (Amazon SQS) queue. Configure an EC2 Reserved Instance to read the messages from the queue and to process the images.

Reserved Instances are typically more expensive than using a Spot Fleet or Lambda for this use case, making it less cost-effective.

- ☐ Use S3 Event Notifications to publish a message with image details to an Amazon Simple Notification Service (Amazon SNS) topic. Configure a container instance in Amazon Elastic Container Service (Amazon ECS) to subscribe to the topic and to process the images.

Container instances in ECS are generally more expensive than Lambda for this task. Also, SNS is not ideal for guaranteed message delivery compared to SQS.

- ☐ Use S3 Event Notifications to publish a message with image details to an Amazon Simple Notification Service (Amazon SNS) topic. Configure an AWS Elastic Beanstalk application to subscribe to the topic and to process the images.

Elastic Beanstalk is generally more expensive than Lambda. Also, SNS is not ideal for guaranteed message delivery compared to SQS.

Overall Explanation:

The most cost-effective solution is to use S3 Event Notifications to trigger an AWS Lambda function via an SQS queue. Lambda provides a serverless, cost-efficient way to process images when they are uploaded. SQS ensures that all messages are delivered and processed, handling any potential load spikes gracefully. The image processing time of 2

minutes and memory requirement of 512MB are well within Lambda's capabilities. EC2 Reserved Instances, ECS, and Elastic Beanstalk are generally more expensive options for this on-demand, short-duration processing task. SNS also isn't the best queueing mechanism, SQS offers guaranteed delivery of messages.

[View Discussions](#)[Ask AI](#)

Question 6

Not Attempted

Networking



A company wants to improve the availability and performance of its hybrid application. The application consists of a stateful TCP-based workload hosted on Amazon EC2 instances in different AWS Regions and a stateless UDP-based workload hosted on premises. Which combination of actions should a solutions architect take to improve availability and performance?

- ☒ Create an accelerator using AWS Global Accelerator. Add the load balancers as endpoints.
AWS Global Accelerator improves application availability and performance by directing traffic to optimal endpoints over the AWS global network. It is well-suited for non-HTTP traffic like UDP.
- ☐ Create an Amazon CloudFront distribution with an origin that uses Amazon Route 53 latency-based routing to route requests to the load balancers.
Amazon CloudFront is designed for HTTP/HTTPS traffic and caching content closer to users. It is not the appropriate service for TCP and UDP traffic.
- ☐ Configure two Application Load Balancers in each Region. The first will route to the EC2 endpoints, and the second will route to the on-premises endpoints.
Application Load Balancers are suitable for HTTP/HTTPS traffic. The TCP workload requires a Network Load Balancer.
- ☒ Configure a Network Load Balancer in each Region to address the EC2 endpoints. Configure a Network Load Balancer in each Region that routes to the on-premises endpoints.
Network Load Balancers (NLBs) are ideal for handling TCP-based traffic and can distribute traffic across multiple Availability Zones within a Region, improving availability.
- ☐ Configure a Network Load Balancer in each Region to address the EC2 endpoints. Configure an Application Load Balancer in each Region that routes to the on-premises

endpoints.

Network Load Balancers (NLB) are ideal for handling TCP-based traffic. Application Load Balancer is not suitable for the UDP-based on-premise workload.

Overall Explanation:

To improve the availability and performance of the hybrid application, the solutions architect should create an accelerator using AWS Global Accelerator (Option A) for the UDP-based on-premises workload. Additionally, Network Load Balancers (NLBs) in each Region (Option D) should be configured to handle the stateful TCP traffic for the EC2 instances, enabling better distribution and failover capabilities.

[View Discussions](#)[Ask AI](#)[Question 7](#)[Not Attempted](#)[Disaster Recovery](#)

A company runs a self-managed Microsoft SQL Server on Amazon EC2 instances and Amazon Elastic Block Store (Amazon EBS). Daily snapshots are taken of the EBS volumes. Recently, all the company's EBS snapshots were accidentally deleted while running a snapshot cleaning script that deletes all expired EBS snapshots. A solutions architect needs to update the architecture to prevent data loss without retaining EBS snapshots indefinitely. Which solution will meet these requirements with the LEAST development effort?

- ☐ ☒ Change the IAM policy of the user to deny EBS snapshot deletion.
Changing the IAM policy prevents deletion by a specific user but doesn't safeguard against other means of accidental deletion. It also requires modifying user policies, which can be complex.
- ☐ ☒ Copy the EBS snapshots to another AWS Region after completing the snapshots daily.
Copying snapshots to another region provides redundancy but involves significant data transfer costs and development effort for managing the copy process.
- ☒ ☐ Create a 7-day EBS snapshot retention rule in Recycle Bin and apply the rule for all snapshots.
AWS Recycle Bin allows setting retention rules for EBS snapshots, preventing permanent deletion for a specified period. This provides a safety net against accidental deletions and is easily configurable.
- ☐ ☒ Copy EBS snapshots to Amazon S3 Standard-Infrequent Access (S3 Standard-IA).

Copying snapshots to S3 incurs storage costs and requires development effort to manage the transfer and lifecycle of snapshots in S3. It also adds complexity compared to Recycle Bin.

Overall Explanation:

Creating a 7-day EBS snapshot retention rule in Recycle Bin offers the least development effort while preventing permanent data loss from accidental deletions. Recycle Bin allows for recovery of accidentally deleted snapshots within the retention period. IAM policy changes are user-specific, cross-region copying is complex and costly, and S3 requires more management overhead compared to Recycle Bin.

[View Discussions](#)[Ask AI](#)

Question 8

Not Attempted

Security



A company wants to use an AWS CloudFormation stack for its application in a test environment. The company stores the CloudFormation template in an Amazon S3 bucket that blocks public access. The company wants to grant CloudFormation access to the template in the S3 bucket based on specific user requests to create the test environment. The solution must follow security best practices. Which solution will meet these requirements?

- ☐ Create a gateway VPC endpoint for Amazon S3. Configure the CloudFormation stack to use the S3 object URL.
A gateway VPC endpoint for S3 provides private access to the S3 template, aligns with security best practices, and incurs no additional costs.
- ☐ Create an Amazon API Gateway REST API that has the S3 bucket as the target. Configure the CloudFormation stack to use the API Gateway URL.
Creating an API Gateway adds unnecessary complexity and cost compared to a presigned URL or VPC endpoint.
- ☒ Create a presigned URL for the template object. Configure the CloudFormation stack to use the presigned URL.
A presigned URL grants temporary access to an S3 object without making it publicly accessible and is based on specific user requests.
- ☐ Allow public access to the template object in the S3 bucket. Block the public access after the test environment is created.

Allowing public access, even temporarily, violates security best practices.

Overall Explanation:

Creating a presigned URL allows CloudFormation to access the template object in S3 for the duration specified by the URL, while maintaining security by not making the object publicly accessible and fulfilling the requirement for access based on specific user requests.

[View Discussions](#)[Ask AI](#)

Question 9

Not Attempted

Security



A company's applications run within an AWS Organizations environment. Operational support is outsourced to external engineers. The company must grant access to these engineers without compromising security. The engineers require AWS Management Console access and OS-level access to Amazon EC2 instances (running Amazon Linux in private subnets). Which solution provides the MOST secure access?

- ✔ Confirm that AWS Systems Manager Agent (SSM Agent) is installed on all instances. Assign an instance profile with the necessary policy to connect to Systems Manager. Use AWS IAM Identity Center to provide the external support engineers console access. Use Systems Manager Session Manager to assign the required permissions.

Leverages SSM Session Manager for secure, auditable access without opening SSH ports or managing SSH keys, reducing attack surface. IAM Identity Center provides federated access to the console.

- ✘ Confirm that AWS Systems Manager Agent (SSM Agent) is installed on all instances. Assign an instance profile with the necessary policy to connect to Systems Manager. Use Systems Manager Session Manager to provide local IAM user credentials in each AWS account to the external support engineers for console access.

Providing local IAM user credentials increases operational overhead and security risks. Managing these credentials across multiple AWS accounts can lead to inconsistency and potential vulnerabilities.

- ✘ Confirm that all instances have a security group that allows SSH access only from the external support engineers' source IP address ranges. Provide local IAM user credentials in each AWS account to the external support engineers for console access. Provide each external support engineer an SSH key pair to log in to the application instances.

Allowing SSH access via security groups and distributing SSH key pairs introduces significant security risks. Keys could be lost or misused, and managing IP address ranges is cumbersome.

- ⊗ Create a bastion host in a public subnet. Set up the bastion host security group to allow access from only the external engineers' IP address ranges. Ensure that all instances have a security group that allows SSH access from the bastion host. Provide each external support engineer an SSH key pair to log in to the application instances. Provide local account IAM user credentials to the engineers for console access.

Bastion hosts require ongoing maintenance and monitoring. Using SSH keys introduces the risk of key mismanagement, and using local IAM user credentials increases the attack surface.

Overall Explanation:

The most secure solution is to use Systems Manager Session Manager for EC2 instance access and AWS IAM Identity Center for console access. Session Manager eliminates the need for SSH keys and open ports, reducing the attack surface. IAM Identity Center provides centralized access management and avoids the security risks associated with local IAM users.

[View Discussions](#)[Ask AI](#)

Question 10

Not Attempted

Database



A company runs its applications on Amazon RDS for PostgreSQL in the us-east-1 Region. The company also uses machine learning (ML) models to forecast annual revenue based on near real-time reports. These reports are generated using the same RDS for PostgreSQL database. During business hours, the database experiences performance slowdowns. The company wants to improve database performance cost-effectively. Which solution should the company use?

- ⊗ Create a cross-Region read replica. Configure the reports to be generated from the read replica.

Creating a cross-Region read replica will add latency due to geographic distance and incur higher data transfer costs. It's not the most cost-effective solution.

- ⊗ Activate Multi-AZ DB instance deployment for RDS for PostgreSQL. Configure the reports to be generated from the standby database.

Multi-AZ is primarily for high availability and failover. The standby database is not designed to handle read requests. Also, it doesn't offload the reporting workload from the primary instance for

performance improvement.

- ⊗ Use AWS Data Migration Service (AWS DMS) to logically replicate data to a new database. Configure the reports to be generated from the new database.

Using DMS to replicate to a new database is more complex and potentially more expensive than a read replica. It involves managing a separate DMS task and database, adding unnecessary overhead.

- ✓ Create a read replica in us-east-1. Configure the reports to be generated from the read replica.

Creating a read replica in the same region and directing report generation to it is the most cost-effective way to offload the reporting workload from the primary database instance, improving its performance during business hours. It avoids cross-region data transfer costs and complexity.

Overall Explanation:

Creating a read replica in the same region (us-east-1) and configuring the reports to be generated from the read replica is the most cost-effective solution. It offloads the reporting workload from the primary database, improving its performance during business hours, without incurring cross-region data transfer costs or unnecessary complexity. Cross-region replicas are more costly and add latency. Multi-AZ deployment's standby isn't for read replicas and does not remove the load from the original DB. AWS DMS adds unnecessary overhead for this purpose.

[View Discussions](#)[Ask AI](#)

Question 11

Not Attempted

Networking



A company hosts applications on Amazon EC2 instances within a VPC. These applications need to interact with Amazon S3 to store and retrieve objects. The company's security policies strictly prohibit any application traffic from traversing the public internet. Which solution ensures the EC2 instances can securely access S3 without internet access?

- ✓ Configure an S3 gateway endpoint.

A gateway endpoint allows direct and secure connectivity to S3 from within the VPC, ensuring traffic doesn't traverse the public internet. This is the correct solution.

- ☐ Create an S3 bucket in a private subnet.
S3 buckets do not reside in subnets, so this option is invalid.
- ☐ Create an S3 bucket in the same AWS Region as the EC2 instances.
Creating the bucket in the same region doesn't inherently prevent traffic from traversing the internet. Additional configuration is required for secure, private connectivity.
- ☐ Configure a NAT gateway in the same subnet as the EC2 instances.
A NAT gateway provides outbound internet connectivity for resources in private subnets, which violates the requirement of preventing traffic from traversing the internet.

Overall Explanation:

An S3 gateway endpoint provides secure and reliable connectivity to Amazon S3 from within your VPC without requiring an internet gateway or NAT device. This ensures that traffic between the EC2 instances and S3 remains within the AWS network and does not traverse the internet, meeting the company's security requirements.

[View Discussions](#)[Ask AI](#)

Question 12

Not Attempted

[Monitoring and Logging](#)

A company hosts a multi-tier, public web application in the AWS Cloud. The web application runs on Amazon EC2 instances, and its database runs on Amazon RDS. The company is anticipating a large increase in sales during an upcoming holiday weekend. A solutions architect needs to build a solution to analyze the performance of the web application with a granularity of no more than 2 minutes. What should the solutions architect do to meet this requirement?

- ☐ Send Amazon CloudWatch logs to Amazon Redshift. Use Amazon QuickSight to perform further analysis.
Sending CloudWatch logs to Redshift for analysis with QuickSight would introduce latency and complexity, and is not the most efficient solution for near real-time monitoring with the required granularity.
- ☒ Enable detailed monitoring on all EC2 instances. Use Amazon CloudWatch metrics to perform further analysis.

Enabling detailed monitoring on EC2 instances provides metrics at a 1-minute granularity, which meets the requirement of analyzing performance with a granularity of no more than 2 minutes. CloudWatch metrics can be used directly for analysis and visualization.

- ⊗ Create an AWS Lambda function to fetch EC2 logs from Amazon CloudWatch Logs. Use Amazon CloudWatch metrics to perform further analysis.

Using a Lambda function to fetch EC2 logs from CloudWatch Logs adds unnecessary complexity. CloudWatch metrics already provide the required granularity without needing to process logs in this way.

- ⊗ Send EC2 logs to Amazon S3. Use Amazon Redshift to fetch logs from the S3 bucket to process raw data for further analysis with Amazon QuickSight.

Sending EC2 logs to S3 and then using Redshift and QuickSight is overkill for this scenario. It introduces unnecessary complexity and latency. CloudWatch metrics offer a more direct and efficient way to monitor performance.

Overall Explanation:

The most efficient and effective way to meet the requirement is to enable detailed monitoring on the EC2 instances. This provides 1-minute granularity metrics in CloudWatch, which can be used for near real-time performance analysis. Other options introduce unnecessary complexity and latency.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

Serverless



A company hosts a photo-sharing application where users upload photos directly to an Amazon S3 bucket. On average, 150 photos are uploaded daily. The company wants to automatically generate a thumbnail for each new photo and store it in a separate S3 bucket. Which solution offers the MOST cost-effective approach for accomplishing this task?

- ⊗ Configure an Amazon EventBridge scheduled rule to invoke a script every minute on a long-running Amazon EMR cluster. Configure the script to generate thumbnails for the photos that do not have thumbnails. Configure the script to upload the thumbnails to the second S3 bucket.

Using EMR for this task is highly inefficient and expensive as it's designed for big data processing, not real-time thumbnail generation.

- ⊗ Configure an Amazon EventBridge scheduled rule to invoke a script every minute on a memory-optimized Amazon EC2 instance that is always on. Configure the script to generate thumbnails for the photos that do not have thumbnails. Configure the script to upload the thumbnails to the second S3 bucket.

While feasible, maintaining a constantly running EC2 instance introduces unnecessary operational overhead and costs compared to a serverless approach.

- ✔ Configure an S3 event notification to invoke an AWS Lambda function each time a user uploads a new photo to the application. Configure the Lambda function to generate a thumbnail and to upload the thumbnail to the second S3 bucket.

This is the most cost-effective solution. Lambda functions are event-driven and only execute when a new photo is uploaded, minimizing costs. S3 event notifications seamlessly trigger the Lambda function.

- ⊗ Configure S3 Storage Lens to invoke an AWS Lambda function each time a user uploads a new photo to the application. Configure the Lambda function to generate a thumbnail and to upload the thumbnail to a second S3 bucket.

S3 Storage Lens is an analytics service and is not designed to trigger Lambda functions based on individual object uploads. Using it for this purpose would be inappropriate and ineffective.

Overall Explanation:

The most cost-effective solution is to use S3 event notifications to trigger an AWS Lambda function. Lambda is serverless and only runs when a new photo is uploaded. This eliminates the need for constantly running instances and minimizes costs. EMR is designed for Big Data and is an overkill for thumbnails generation. S3 Storage Lens is a storage analytics feature and cannot trigger Lambda functions for individual file uploads.

 View Discussions

 Ask AI

Question 14

Not Attempted

Serverless



A company stores millions of objects in an Amazon S3 bucket, using S3 Glacier Deep Archive storage class across multiple prefixes. The company must delete data older than 3 years, but a specific subset of data must be retained. The company has identified this retained data and wants a serverless solution for the deletion process. Which solution meets these requirements in the MOST cost-effective and operationally efficient manner?

- ⊗ Use S3 Inventory to list all objects. Utilize the AWS CLI to create a script that runs on an Amazon EC2 instance to delete objects from the inventory list.

This solution is not serverless, as it relies on a long-running EC2 instance. It is also not the most cost-effective solution.

- ⊗ Use AWS Batch to delete objects older than 3 years, excluding the data to be retained.

AWS Batch is designed for batch computing workloads and is not the best fit for managing and deleting S3 objects. It will be more complex than necessary.

- ⊗ Provision an AWS Glue crawler to query objects older than 3 years. Save the manifest file of old objects. Create a script to delete objects in the manifest.

AWS Glue crawlers are better suited for metadata extraction and schema discovery. Using it for object deletion is inefficient and not its primary purpose. It's also not serverless as you have to create a separate script and manage it.

- ✓ Enable S3 Inventory. Create an AWS Lambda function to filter and delete objects. Invoke the Lambda function with S3 Batch Operations to delete objects using the inventory reports.

This is the MOST suitable serverless solution. S3 Inventory provides a report of all objects, the Lambda function can filter objects based on age and retention policies, and S3 Batch Operations allows for bulk deletion based on the Lambda function's output.

Overall Explanation:

The optimal solution involves leveraging S3 Inventory to generate a report of all objects, utilizing an AWS Lambda function to filter objects based on age and retention requirements, and then employing S3 Batch Operations to efficiently delete the identified objects. This approach avoids the need for long-running servers or inappropriate services like AWS Glue, resulting in a cost-effective and operationally efficient serverless solution.

 View Discussions

 Ask AI

Question 15

Not Attempted

Security



A company is developing an application on AWS that uses several AWS Lambda functions to process sensitive data retrieved from a single Amazon S3 bucket. To adhere to security best practices, the company requires that only authorized Lambda functions have access to this data, following the principle of least privilege. Which solution fulfills this requirement?

- ⊗ Grant a single IAM role with full S3 bucket access to all Lambda functions.

Granting full access violates the principle of least privilege, as each Lambda function would have more permissions than necessary.
- ⊗ Configure the Lambda functions to operate within a VPC. Implement a bucket policy that grants access based on the Lambda functions' VPC endpoint IP addresses.

While technically possible, managing access based on IP addresses is complex, less secure, and not recommended for Lambda functions. IAM roles are preferable.
- ✔ Create unique IAM roles for each Lambda function. Grant each IAM role specific access to the S3 bucket. Assign the corresponding IAM role as the execution role for each Lambda function.

This solution aligns with the principle of least privilege. Each Lambda function receives a specific IAM role with only the necessary permissions to access the S3 bucket.
- ⊗ Implement a bucket policy granting access to the Lambda functions based on their function ARNs.

Using bucket policies with function ARNs is a viable solution, but less scalable and flexible than using IAM roles directly attached to the Lambda functions, especially as the number of functions grows.

Overall Explanation:

The best solution is to create individual IAM roles for each Lambda function, granting each role access to the S3 bucket based on the least privilege principle. This allows fine-grained control and minimizes the risk of unauthorized access. While bucket policies with ARNs can work, IAM roles are more scalable and manageable for a larger number of Lambda functions.

[View Discussions](#)[Ask AI](#)

Question 16

Not Attempted

Serverless



A company has developed a non-production application composed of multiple microservices, one set for each of the company's business units. A single development team maintains all the microservices. The current architecture uses a static web frontend and a Java-based backend that contains the application logic. The architecture also uses a MySQL database that the company hosts on an Amazon EC2 instance. The company needs to ensure that the application

is secure and available globally with the LEAST operational overhead. Which solution will meet these requirements?

- ⊗ Use Amazon CloudFront and AWS Amplify to host the static web frontend. Refactor the microservices to use AWS Lambda functions that the microservices access by using Amazon API Gateway. Migrate the MySQL database to an Amazon EC2 Reserved Instance.

Using EC2 Reserved Instance requires management of the instance itself and doesn't provide the scaling benefits of a managed database service. AWS Amplify is a good option, but less cost efficient.

- ✔ Use Amazon CloudFront and Amazon S3 to host the static web frontend. Refactor the microservices to use AWS Lambda functions that the microservices access by using Amazon API Gateway. Migrate the MySQL database to Amazon RDS for MySQL.

This option provides a scalable and highly available solution. Amazon S3 for static content, Lambda and API Gateway for the backend logic, and RDS for MySQL offer a serverless architecture that minimizes operational overhead and ensures global availability through CloudFront.

- ⊗ Use Amazon CloudFront and Amazon S3 to host the static web frontend. Refactor the microservices to use AWS Lambda functions that are in a target group behind a Network Load Balancer. Migrate the MySQL database to Amazon RDS for MySQL.

Using a Network Load Balancer (NLB) with Lambda functions is not a typical pattern. API Gateway is the preferred way to expose Lambda functions as an API. NLB is better suited for long running TCP Connections.

- ⊗ Use Amazon S3 to host the static web frontend. Refactor the microservices to use AWS Lambda functions that are in a target group behind an Application Load Balancer. Migrate the MySQL database to an Amazon EC2 Reserved Instance.

Using an EC2 Reserved Instance requires management of the instance itself and doesn't provide the scaling benefits of a managed database service. Using an Application Load Balancer (ALB) with Lambda functions is possible, but API Gateway is more suitable and costs less. S3 hosting alone does not provide global distribution like CloudFront.

Overall Explanation:

The optimal solution involves using Amazon CloudFront with Amazon S3 for static content hosting, refactoring the microservices into AWS Lambda functions accessed through Amazon API Gateway, and migrating the MySQL database to Amazon RDS for MySQL. This provides global availability, security, and minimizes operational overhead by leveraging managed services. Therefore, option B is the correct answer.

[View Discussions](#)[Ask AI](#)

Question 17

Not Attempted

Database



A video game company is deploying a new gaming application to its global users. The company requires a solution that will provide near real-time reviews and rankings of the players. A solutions architect must design a solution to provide fast access to the data. The solution must also ensure the data persists on disks in the event that the company restarts the application. Which solution will meet these requirements with the LEAST operational overhead?

- ☐ Configure an Amazon CloudFront distribution with an Amazon S3 bucket as the origin. Store the player data in the S3 bucket.

Amazon CloudFront and S3 are primarily for content delivery and durable storage, not for real-time data updates and low-latency access for ranking and reviews. S3 is not designed for the low-latency reads and writes required for near real-time ranking.

- ☐ Create Amazon EC2 instances in multiple AWS Regions. Store the player data on the EC2 instances. Configure Amazon Route 53 with geolocation records to direct users to the closest EC2 instance.

While this setup could potentially achieve low latency via geographically distributed EC2 instances, managing the EC2 instances, data replication, and Route 53 configurations would result in significantly higher operational overhead than a managed caching solution.

- ☒ Deploy an Amazon ElastiCache for Redis cluster. Store the player data in the ElastiCache cluster.

Amazon ElastiCache for Redis provides in-memory caching with low latency and high throughput, which is ideal for near real-time access to player reviews and rankings. Redis supports data persistence to disk (RDB snapshots and AOF), ensuring data is preserved across application restarts. ElastiCache reduces the operational overhead because it is a managed service.

- ☐ Deploy an Amazon ElastiCache for Memcached cluster. Store the player data in the ElastiCache cluster.

Amazon ElastiCache for Memcached provides in-memory caching for low latency; however, Memcached does not offer built-in persistence, which is a key requirement.

Overall Explanation:

ElastiCache for Redis cluster is the best solution. It offers in-memory caching for low latency reads/writes, supports data persistence, and has lower operational overhead

compared to EC2-based solutions. Redis with AOF (Append-Only File) persistence will ensure data durability. While Memcached is a caching solution, it does not provide data persistence.

[View Discussions](#)[Ask AI](#)

Question 18

Not Attempted

Security



A company is designing an application on AWS that processes sensitive data. The application stores and processes financial data for multiple customers. To meet compliance requirements, the data for each customer must be encrypted separately at rest by using a secure, centralized key management solution. The company wants to use AWS Key Management Service (AWS KMS) to implement encryption. Which solution will meet these requirements with the LEAST operational overhead?

- ☐ Generate a unique encryption key for each customer. Store the keys in an Amazon S3 bucket. Enable server-side encryption.

Storing keys in S3, even with server-side encryption, introduces operational overhead and doesn't provide the same level of control and auditing as AWS KMS.

- ☐ Deploy a hardware security appliance in the AWS environment that securely stores customer-provided encryption keys. Integrate the security appliance with AWS KMS to encrypt the sensitive data in the application.

Deploying and managing a hardware security appliance introduces significant operational overhead compared to using AWS KMS directly.

- ☐ Create a single AWS KMS key to encrypt all sensitive data across the application.

Using a single key for all customers does not meet the requirement of encrypting each customer's data separately.

- ☒ Create separate AWS KMS keys for each customer's data that have granular access control and logging enabled.

Creating separate KMS keys for each customer provides the required isolation and control, while KMS simplifies key management and reduces operational overhead. Granular access control and logging enhance security and auditing.

Overall Explanation:

Creating separate AWS KMS keys for each customer's data with granular access control and logging enabled meets the requirement of encrypting each customer's data separately, providing a secure, centralized key management solution with minimal operational overhead. Storing keys in S3 or using a hardware security appliance increases operational complexity. A single KMS key does not meet the separation requirement.

[View Discussions](#)[Ask AI](#)

Question 19

Not Attempted

[Compute](#)

A company needs to design a resilient web application to process customer orders. The web application must automatically handle increases in web traffic and application usage without affecting the customer experience or losing customer orders. Which solution will meet these requirements?

- ☐ Use a NAT gateway to manage web traffic. Use Amazon EC2 Auto Scaling groups to receive, process, and store processed customer orders. Use an AWS Lambda function to capture and store unprocessed orders.

NAT Gateway is not suitable for managing web traffic. Lambda is suitable for event-driven tasks, not for capturing all unprocessed orders directly.

- ☐ Use a Network Load Balancer (NLB) to manage web traffic. Use an Application Load Balancer to receive customer orders from the NLB. Use Amazon Redshift with a Multi-AZ deployment to store unprocessed and processed customer orders.

While NLB can manage traffic, Redshift is primarily for data warehousing, not for transactional order processing.

- ☐ Use a Gateway Load Balancer (GWLB) to manage web traffic. Use Amazon Elastic Container Service (Amazon ECS) to receive and process customer orders. Use the GWLB to capture and store unprocessed orders. Use Amazon DynamoDB to store processed customer orders.

GWLB is typically used for virtual appliances, not for general web traffic management. Capturing unprocessed orders directly by GWLB is not a standard practice.

- ☒ Use an Application Load Balancer to manage web traffic. Use Amazon EC2 Auto Scaling groups to receive and process customer orders. Use Amazon Simple Queue Service (Amazon SQS) to store unprocessed orders. Use Amazon RDS with a Multi-AZ deployment to store processed customer orders.

Application Load Balancer effectively distributes web traffic. Auto Scaling groups ensure scalability. SQS provides a reliable queue for unprocessed orders. RDS with Multi-AZ deployment ensures high availability for storing processed orders.

Overall Explanation:

Using an Application Load Balancer (ALB) allows for intelligent traffic distribution. EC2 Auto Scaling groups provide scalability for handling fluctuating traffic. Amazon SQS acts as a buffer for unprocessed orders, preventing data loss during traffic spikes. Amazon RDS with Multi-AZ provides durability for storing processed orders. This combination ensures resilience and scalability.

[View Discussions](#)[Ask AI](#)

Question 20

Not Attempted

Storage



A company is migrating millions of files, averaging 10 KB in size, from on-premises to AWS using DataSync. They plan to store the files in Amazon S3. For the first year, the files will be accessed infrequently (once or twice) and need to be immediately available. After one year, the files will be archived for a minimum of seven years. What solution provides the MOST cost-effective approach?

- ⊗ Use an archive tool to group the files into large objects. Use DataSync to migrate the objects. Store the objects in S3 Glacier Instant Retrieval for the first year. Use a lifecycle configuration to transition the files to S3 Glacier Deep Archive after 1 year with a retention period of 7 years.

Grouping files adds unnecessary complexity. S3 Glacier Instant Retrieval may be more expensive than S3 Standard-IA for infrequently accessed data during the first year, and has a minimum object size limit that would make it unsuitable for 10KB files.

- ⊗ Use an archive tool to group the files into large objects. Use DataSync to copy the objects to S3 Standard-Infrequent Access (S3 Standard-IA). Use a lifecycle configuration to transition the files to S3 Glacier Instant Retrieval after 1 year with a retention period of 7 years.

Grouping files adds unnecessary complexity, and transitioning to S3 Glacier Instant Retrieval after one year doesn't make sense as that storage class is intended for archival from the beginning. Additionally, S3 Glacier Instant Retrieval is also not the most cost effective long term archival option. S3 Standard-IA also has a minimum object size limit.

- ⊗ Configure the destination storage class for the files as S3 Glacier Instant Retrieval. Use a lifecycle policy to transition the files to S3 Glacier Flexible Retrieval after 1 year with a retention period of 7 years.

S3 Glacier Instant Retrieval is designed for archival, not general-purpose immediate availability. It also has a minimum object size of 128KB, making it unsuitable for the many 10KB files. S3 Glacier Flexible Retrieval (formerly S3 Glacier) is not the lowest cost archival option, thus not most cost-effective.

- ✔ Configure a DataSync task to transfer the files to S3 Standard-Infrequent Access (S3 Standard-IA). Use a lifecycle configuration to transition the files to S3 Glacier Deep Archive after 1 year with a retention period of 7 years.

S3 Standard-IA offers immediate availability at a lower cost than Glacier Instant Retrieval when data is accessed infrequently. Transitioning to S3 Glacier Deep Archive provides the lowest-cost long-term storage. S3 Standard-IA has a minimum object size of 128KB.

Overall Explanation:

Option D is the most cost-effective solution. Storing directly in S3 Standard-IA provides immediate availability with infrequent access and then moving the files to S3 Glacier Deep Archive after one year provides low cost archival storage. Options A and B add complexity with the use of archiving tools that are not needed and they are not cost-effective. Option C uses Glacier Instant Retrieval which has a higher price than S3 Standard IA for infrequently accessed data.

 View Discussions

 Ask AI

Question 21

Not Attempted

Storage



A company migrated its on-premises Oracle database to an Amazon EC2 memory-optimized Linux instance. The instance uses a 1 TB Provisioned IOPS SSD (io1) EBS volume with 64,000 IOPS. After the migration, database storage performance is slower than the on-premises performance. What can improve storage performance?

- ✔ Add more Provisioned IOPS SSD (io1) EBS volumes. Use OS commands to create a Logical Volume Management (LVM) stripe.

Adding more io1 volumes and striping them with LVM allows you to exceed the 64,000 IOPS limit of a single io1 volume, increasing aggregate throughput and performance.

- ⊗ Increase the Provisioned IOPS SSD (io1) EBS volume to more than 64,000 IOPS.
The maximum provisioned IOPS for a single io1 volume is 64,000. You cannot increase it beyond this limit.
- ⊗ Increase the size of the Provisioned IOPS SSD (io1) EBS volume to 2 TB.
Increasing the size alone will not improve IOPS if the application is limited by it. It will only increase available storage capacity.
- ⊗ Change the EC2 Linux instance to a storage-optimized instance type. Do not change the Provisioned IOPS SSD (io1) EBS volume.
Changing the instance type alone may not solve the storage performance bottleneck if the root cause is the IOPS limit of the EBS volume. Storage-optimized instances provide faster networking to EBS, but if you are already at the IOPS limit, the network speed is not the issue. While storage-optimized instance types can improve performance, the primary bottleneck is likely the IOPS limit on the EBS volume.

Overall Explanation:

The performance bottleneck is caused by the maximum IOPS limit of 64,000 on the single io1 EBS volume. Adding more io1 volumes and striping them with LVM allows the application to exceed this limit, thus improving the aggregate storage performance.

Options B, C, and D do not address the fundamental IOPS limitation. Therefore, Option A is the correct answer.

[View Discussions](#)[Ask AI](#)

Question 22

Not Attempted

Security



A company stores sensitive user information in an Amazon S3 bucket. The company requires secure access to this bucket from the application tier running on Amazon EC2 instances within a VPC. Which combination of steps should a solutions architect take to accomplish this? (Choose two.)

- ✔ Configure a VPC gateway endpoint for Amazon S3 within the VPC.
A VPC gateway endpoint provides private connectivity to S3 without requiring internet access, keeping traffic within the AWS network and enhancing security.
- ⊗ Create a bucket policy to make the objects in the S3 bucket public.

Making the S3 bucket public exposes sensitive data to the internet, which contradicts the requirement for secure access.

✔ Create a bucket policy that limits access to only the application tier running in the VPC.
A bucket policy restricting access to the specific VPC or EC2 instances ensures that only the authorized application tier can access the sensitive data.

✘ Create an IAM user with an S3 access policy and copy the IAM credentials to the EC2 instance.

Storing IAM credentials on EC2 instances is a security risk. If the instance is compromised, the credentials could be exposed.

✘ Create a NAT instance and have the EC2 instances use the NAT instance to access the S3 bucket.

A NAT instance allows EC2 instances in a private subnet to access the internet, but it doesn't inherently restrict access to specific resources or provide secure access compared to a VPC endpoint.

Overall Explanation:

To provide secure access to the S3 bucket from EC2 instances within a VPC, a solutions architect should configure a VPC gateway endpoint for S3 (Option A) to enable private connectivity. Additionally, a bucket policy should be created to limit access to only the application tier running in the VPC (Option C), ensuring that only authorized instances can access the sensitive data. Options B and D introduce security risks, while Option E does not provide the necessary level of access control.

 View Discussions

 Ask AI

Question 23

Not Attempted

Serverless



A company is migrating from a monolithic architecture for a web application hosted on Amazon EC2 to a serverless microservices architecture. The company aims to use AWS services supporting an event-driven, loosely coupled architecture, implementing the publish/subscribe (pub/sub) pattern. Which solution meets these requirements MOST cost-effectively?

✘ Configure an Amazon API Gateway REST API to invoke an AWS Lambda function that publishes events to an Amazon Simple Queue Service (Amazon SQS) queue. Configure

one or more subscribers to read events from the SQS queue.

Using SQS doesn't directly support the pub/sub pattern as efficiently as SNS. SQS is more suited for queuing messages for processing, not broadcasting them.

- ⊗ Configure an Amazon API Gateway REST API to invoke an AWS Lambda function that publishes events to an Amazon Simple Notification Service (Amazon SNS) topic. Configure one or more subscribers to receive events from the SNS topic.

While this uses SNS which is appropriate for pub/sub, the REST API is more expensive than the HTTP API.

- ⊗ Configure an Amazon API Gateway WebSocket API to write to a data stream in Amazon Kinesis Data Streams with enhanced fan-out. Configure one or more subscribers to receive events from the data stream.

Kinesis Data Streams is more suitable for streaming data, not necessarily the most cost-effective solution for a simple pub/sub pattern.

- ✓ Configure an Amazon API Gateway HTTP API to invoke an AWS Lambda function that publishes events to an Amazon Simple Notification Service (Amazon SNS) topic. Configure one or more subscribers to receive events from the topic.

This option offers a cost-effective solution using SNS for pub/sub with a cheaper HTTP API Gateway.

Overall Explanation:

The most cost-effective solution uses an HTTP API Gateway to invoke a Lambda function, which then publishes events to an SNS topic. SNS is designed for pub/sub, providing loose coupling and event-driven architecture. HTTP APIs are generally cheaper than REST APIs. SQS is less suited for pub/sub than SNS, and Kinesis Data Streams is more appropriate for high-volume streaming use cases, making it less cost-effective for simple event notifications.

 View Discussions

 Ask AI

Question 24

Not Attempted

Database



A company recently migrated a monolithic application to an Amazon EC2 instance and Amazon RDS. The application has tightly coupled modules and is designed to run on only a single EC2 instance. During peak usage, the EC2 instance experiences high CPU utilization, leading to degraded read performance on Amazon RDS. The company aims to reduce the CPU utilization

and improve read request performance without modifying the application architecture. Which solution meets these requirements?

- ☒ Resize the EC2 instance to an EC2 instance type with more CPU capacity. Configure an Auto Scaling group with a minimum and maximum size of 1. Configure an RDS read replica for read requests.

This approach addresses both the high CPU utilization by increasing the instance size and the degraded read performance by offloading read requests to an RDS read replica. The Auto Scaling group ensures the instance is always running. This is the most effective solution without changing architecture.

- ☐ Resize the EC2 instance to an EC2 instance type with more CPU capacity. Configure an Auto Scaling group with a minimum and maximum size of 1. Add an RDS read replica and redirect all read/write traffic to the replica.

This option is incorrect because RDS read replicas are designed for read operations only. Redirecting write traffic to the read replica would cause errors and data inconsistencies.

- ☐ Configure an Auto Scaling group with a minimum size of 1 and maximum size of 2. Resize the RDS DB instance to an instance type that has more CPU capacity.

This option only addresses the RDS performance issue by increasing the database instance size. It doesn't directly address the high CPU utilization on the EC2 instance which is the root cause of the read performance degradation. It also requires application changes to take advantage of the additional EC2 instance and is not suitable for a monolithic application.

- ☐ Resize the EC2 instance to an EC2 instance type with more CPU capacity. Configure an Auto Scaling group with a minimum and maximum size of 1. Resize the RDS DB instance to an instance type that has more CPU capacity.

While resizing both EC2 and RDS instances improves performance, it does not explicitly address the read load on the primary RDS instance. Offloading read requests to a read replica, as in Option A, provides a more effective solution for improving read performance.

Overall Explanation:

The best solution is to resize the EC2 instance to handle the CPU load and add an RDS read replica to offload read requests. Resizing the EC2 instance directly reduces CPU bottleneck. Configuring the Auto Scaling group to a single instance preserves the monolithic architecture. Adding a read replica specifically addresses the slow read times by distributing the read load. Other options either try to redirect write operations to read

replicas or doesn't address one of the constraints such as not changing the monolithic architecture.

[View Discussions](#)[Ask AI](#)

Question 25

Not Attempted

Security



A company needs to grant a team of developers access to the company's AWS resources while maintaining a high level of security and preventing unauthorized access to sensitive data.

Which access control solution best meets these requirements?

- ☐ Share the IAM user credentials for each development team member with the rest of the team to simplify access management and to streamline development workflows.

Sharing IAM user credentials is a security risk and violates AWS best practices. It does not adhere to the principle of least privilege and makes auditing impossible.

- ☒ Define IAM roles that have fine-grained permissions based on the principle of least privilege. Assign an IAM role to each developer.

IAM roles allow for temporary credentials that can be automatically rotated, improving security. Defining fine-grained permissions based on the principle of least privilege ensures that developers only have access to the resources they need.

- ☐ Create IAM access keys to grant programmatic access to AWS resources. Allow only developers to interact with AWS resources through API calls by using the access keys.

Managing static access keys for multiple developers introduces significant operational overhead and security risks. If a key is compromised, it can be difficult to revoke access quickly.

- ☐ Create an AWS Cognito user pool. Grant developers access to AWS resources by using the user pool.

AWS Cognito is primarily designed for managing end-user authentication (for web or mobile apps or such), not for managing access to AWS resources for internal developers.

Overall Explanation:

Defining IAM roles with fine-grained permissions based on the principle of least privilege is the best approach. IAM roles provide temporary credentials and allow for precise control over which AWS resources developers can access, ensuring a high level of security.

[View Discussions](#)[Ask AI](#)

Question 26

Not Attempted

[Compute](#)

A company hosts a monolithic web application on an Amazon EC2 instance. Application users have recently reported poor performance at specific times. Analysis of Amazon CloudWatch metrics shows that CPU utilization is 100% during the periods of poor performance. The company wants to resolve this performance issue and improve application availability. Which combination of steps will meet these requirements MOST cost-effectively?

- ☒ Use AWS Compute Optimizer to obtain a recommendation for an instance type to scale vertically.

AWS Compute Optimizer analyzes your current EC2 instance usage and recommends the most cost-effective instance type. Scaling vertically (upgrading to a larger instance type) could provide immediate relief from the 100% CPU utilization.

- ☐ Create an Amazon Machine Image (AMI) from the web server. Reference the AMI in a new launch template.

Creating an AMI and launch template are prerequisites for horizontal scaling but don't directly address the immediate CPU utilization issue or guarantee cost-effectiveness on their own.

- ☐ Create an Auto Scaling group and an Application Load Balancer to scale vertically.

Auto Scaling groups and Application Load Balancers are used for horizontal scaling, not vertical scaling.

- ☐ Use AWS Compute Optimizer to obtain a recommendation for an instance type to scale horizontally.

Horizontal scaling involves adding more instances, not changing the instance type with Compute Optimizer recommendations, even though recommendations can be used for horizontal scaling.

- ☒ Create an Auto Scaling group and an Application Load Balancer to scale horizontally.

This allows the application to scale out during peak times, distributing the load across multiple instances and improving availability. It's cost-effective as it only uses additional resources when needed.

Overall Explanation:

The most cost-effective approach is to first scale vertically by using AWS Compute Optimizer to find a larger, more capable instance to handle the workload. Then, to ensure

continued availability and handle future peaks, implement horizontal scaling with an Auto Scaling group and Application Load Balancer. This ensures resources are only used when needed, optimizing cost.

[View Discussions](#)[Ask AI](#)

Question 27

Not Attempted

Security



A company runs all its business applications in the AWS Cloud. The company uses AWS Organizations to manage multiple AWS accounts. A solutions architect needs to review all permissions that are granted to IAM users to determine which IAM users have more permissions than required. Which solution will meet these requirements with the LEAST administrative overhead?

- ☐ Use Network Access Analyzer to review all access permissions in the company's AWS accounts.
Network Access Analyzer is primarily designed for network configurations and not for identifying excessive IAM permissions.
- ☐ Create an AWS CloudWatch alarm that activates when an IAM user creates or modifies resources in an AWS account.
CloudWatch alarms based on resource creation/modification won't provide insights into whether permissions are overly broad; it only detects actions.
- ☒ Use AWS Identity and Access Management (IAM) Access Analyzer to review all the company's resources and accounts.
IAM Access Analyzer is specifically designed to identify resources shared with external entities and also helps to identify unused access.
- ☐ Use Amazon Inspector to find vulnerabilities in existing IAM policies.
Amazon Inspector focuses on finding vulnerabilities in applications and infrastructure configurations, not IAM permission analysis.

Overall Explanation:

IAM Access Analyzer directly addresses the problem of identifying IAM users with more permissions than needed. It helps to identify unused access and provides insights into potentially overly permissive configurations across all accounts within the AWS

Organization, with minimal administrative overhead. Therefore, it's the most suitable choice.

[View Discussions](#)[Ask AI](#)

Question 28

Not Attempted

Storage



A company must comply with regulatory requirements to ensure that sensitive documents stored in an Amazon S3 bucket cannot be deleted or modified for a specific retention period. What is the MOST suitable solution to meet this compliance requirement?

- ☐ Activate S3 Object Lock on the required objects and enable governance mode.

Governance mode allows users with specific IAM permissions to bypass the object lock. This does not provide the strongest protection against deletion or modification required for strict regulatory compliance.

- ☒ Activate S3 Object Lock on the required objects and enable compliance mode.

Compliance mode offers the strongest level of protection. Once an object is locked in compliance mode, not even the root user can delete or modify the object during the retention period. This directly addresses the regulatory compliance requirement of preventing deletion or modification.

- ☐ Enable versioning on the S3 bucket. Set a lifecycle policy to delete the objects after a specified period.

While versioning protects against accidental deletion by allowing you to recover previous versions, a lifecycle policy to delete objects would violate the requirement to preserve them for a specific period. Also, versioning alone doesn't prevent modification or deletion by authorized users.

- ☐ Configure an S3 Lifecycle policy to transition objects to S3 Glacier Flexible Retrieval for the retention duration.

Transitioning objects to S3 Glacier Flexible Retrieval is useful for cost optimization of infrequently accessed data, but it does not prevent deletion or modification of the objects. It simply moves them to a different storage class. Lifecycle policies alone do not provide the required protection.

Overall Explanation:

S3 Object Lock in compliance mode is the most suitable solution because it prevents any user, including the root user, from deleting or modifying an object for the specified retention period. This ensures that the data is protected and meets strict regulatory compliance requirements. While lifecycle policies help with cost management and

transitioning data between storage classes, they don't prevent deletion or modification. Versioning alone doesn't strictly prevent object removal.

[View Discussions](#)[Ask AI](#)

Question 29

Not Attempted

Compute



A company runs its customer-facing web application on containers using Amazon Elastic Container Service (Amazon ECS) on AWS Fargate. The web application is resource intensive. The web application needs to be available 24 hours a day, 7 days a week for customers. The company expects the application to experience short bursts of high traffic. The workload must be highly available. Which solution will meet these requirements MOST cost-effectively?

- ☐ Configure an ECS capacity provider with Fargate. Conduct load testing by using a third-party tool. Rightsize the Fargate tasks in Amazon CloudWatch.

This option involves using Fargate and rightsizing tasks but does not explicitly address cost optimization beyond initial sizing or offer a strategy for burst traffic cost reduction. Load testing helps determine appropriate sizing, but CloudWatch alone doesn't automatically rightsize.

- ☐ Configure an ECS capacity provider with Fargate for steady state and Fargate Spot for burst traffic.

This option leverages Fargate for steady-state tasks to ensure high availability and reliability, and Fargate Spot for burst tasks to handle high traffic at a reduced cost. However, using Fargate Spot might cause interruptions and may not meet the requirement of 24/7 availability.

- ☐ Configure an ECS capacity provider with Fargate Spot for steady state and Fargate for burst traffic.

This is not a good approach because Fargate Spot is cheaper but interruptible, and steady state requires higher availability than burst traffic. Fargate is more reliable for steady state but more expensive than Spot, making it a poor choice to use for burst traffic in the context of cost optimization.

- ☒ Configure an ECS capacity provider with Fargate. Use AWS Compute Optimizer to rightsize the Fargate task.

This option uses Fargate for continuous availability and leverages AWS Compute Optimizer to rightsize the Fargate task, ensuring the service is running with cost-effective resources. The Compute Optimizer provides recommendations for Fargate Tasks and takes into account performance characteristics and historical resource utilization.

Overall Explanation:

The most cost-effective solution is to configure an ECS capacity provider with Fargate and use AWS Compute Optimizer to rightsize the Fargate task. Using Fargate ensures continuous availability. The Compute Optimizer supports recommendations for Fargate Tasks based on metrics and analysis. Fargate Spot is not appropriate due to the requirement for 24/7 availability.

[View Discussions](#)[Ask AI](#)

Question 30

Not Attempted

Security



A company is building an application in the AWS Cloud. The application is hosted on Amazon EC2 instances behind an Application Load Balancer (ALB). The company uses Amazon Route 53 for DNS services. The company requires a managed solution with proactive engagement to protect against DDoS attacks. Which solution will meet these requirements?

- ☐ Enable AWS Config. Configure an AWS Config managed rule that detects DDoS attacks.
AWS Config is primarily for configuration management and compliance, not proactive DDoS protection.
- ☐ Enable AWS WAF on the ALB. Create an AWS WAF web ACL with rules to detect and prevent DDoS attacks. Associate the web ACL with the ALB.
AWS WAF can help mitigate some DDoS attacks, but it doesn't provide the managed solution with proactive engagement that AWS Shield Advanced offers.
- ☐ Store the ALB access logs in an Amazon S3 bucket. Configure Amazon GuardDuty to detect and take automated preventative actions for DDoS attacks.
Amazon GuardDuty is a threat detection service that monitors for malicious activity and unauthorized behavior, but it does not proactively engage to mitigate DDoS attacks.
- ☒ Subscribe to AWS Shield Advanced. Configure hosted zones in Route 53. Add ALB resources as protected resources.
AWS Shield Advanced provides enhanced DDoS protection with 24/7 support, cost protection for scaling due to attacks, and more detailed attack analytics, satisfying the requirements.

Overall Explanation:

AWS Shield Advanced offers a managed solution with proactive engagement to detect and mitigate against DDoS attacks. Configuring hosted zones in Route 53 and adding ALB resources as protected resources allows Shield Advanced to actively monitor and protect the application. WAF handles SQL injection and Shield handles DDOS.

[View Discussions](#)[Ask AI](#)

Question 31

Not Attempted

Security



A company hosts a video streaming web application in a VPC. The company uses a Network Load Balancer (NLB) to handle TCP traffic for real-time data processing. There have been unauthorized attempts to access the application. The company wants to improve application security with minimal architectural change to prevent unauthorized attempts to access the application. Which solution will meet these requirements?

- ☐ Implement a series of AWS WAF rules directly on the NLB to filter out unauthorized traffic.

AWS WAF works at Layer 7 (application layer) and is designed for HTTP/HTTPS traffic. NLBs operate at Layer 4 and do not support WAF directly.

- ☐ Recreate the NLB with a security group to allow only trusted IP addresses.

While NLBs now support security groups, recreating the NLB is not the minimal architectural change required. Security groups are applied to the targets of the NLB (EC2 instances, IP addrs, or ALBs) rather than the NLB itself and filtering by IP address may not be scalable or effective against DDoS attacks.

- ☐ Deploy a second NLB in parallel with the existing NLB configured with a strict IP address allow list.

This option introduces unnecessary complexity and involves a significant architectural change, which contradicts the requirement for minimal change.

- ☒ Use AWS Shield Advanced to provide enhanced DDoS protection and prevent unauthorized access attempts.

AWS Shield Advanced provides enhanced DDoS protection, which addresses unauthorized access attempts that may be caused by DDoS attacks. It can be integrated with NLBs to provide comprehensive protection without requiring significant architectural changes.

Overall Explanation:

The most appropriate solution is to use AWS Shield Advanced. It provides enhanced DDoS protection and can prevent unauthorized access attempts without requiring significant architectural changes. AWS WAF doesn't support NLBs, recreating the NLB or deploying a second NLB is not a minimal architectural change.

[View Discussions](#)[Ask AI](#)

Question 32

Not Attempted

Security



A healthcare company is developing an AWS Lambda function that publishes notifications containing protected health information (PHI) to an encrypted Amazon Simple Notification Service (Amazon SNS) topic. The SNS topic is encrypted using AWS Key Management Service (AWS KMS) customer managed keys. To ensure secure message publishing, which combination of steps should the company take to grant the application the necessary permissions?

- ✓ Create a resource policy for the SNS topic that allows the Lambda function to publish messages to the topic.

This is necessary to grant the Lambda function permission to send messages to the SNS topic. The resource policy should include the Lambda function's ARN and the `sns:Publish` action.

- ✗ Use server-side encryption with AWS KMS keys (SSE-KMS) for the SNS topic instead of customer managed keys.

The question specifies customer-managed keys. Changing the key type is not a permission-related step and is therefore incorrect.

- ✓ Create a resource policy for the encryption key that the SNS topic uses, granting SNS the necessary AWS KMS permissions to use the key for encryption and decryption.

This ensures that SNS has permission to use the KMS key for encryption and decryption operations when publishing messages on behalf of the Lambda function. This is a requirement when using customer-managed KMS keys with SNS.

- ✗ Specify the Lambda function's Amazon Resource Name (ARN) in the SNS topic's resource policy.

While specifying the Lambda function's ARN is important for granting permissions, it's not sufficient on its own. It needs to be combined with the proper actions (like `sns:Publish`) within the SNS resource policy.

- ⊗ Associate an Amazon API Gateway HTTP API with the SNS topic to control access to the topic by using API Gateway resource policies.

This is not required, as the Lambda function directly interacts with the SNS topic and API Gateway is not involved in this scenario.

- ✔ Configure a Lambda execution role that has the necessary IAM permissions to use the customer managed key in AWS KMS.

The Lambda function's execution role needs permissions to allow SNS to encrypt messages on its behalf. These permissions should include kms:Encrypt, kms:Decrypt, and kms:GenerateDataKey (depending on KMS key policy and use case).

Overall Explanation:

To securely publish messages to an encrypted SNS topic using customer-managed KMS keys, you need the following: (1) A resource policy on the SNS topic allowing the Lambda function to publish messages. (2) A resource policy on the KMS key allowing SNS to use the key for encryption and decryption. (3) A Lambda execution role with the correct IAM permissions to allow SNS to use the KMS key on behalf of the Lambda function. Options A, C and F are correct.

 View Discussions

 Ask AI

Question 33

Not Attempted

Database



A company hosts an on-premises application with a MySQL database. They're migrating to AWS for elasticity and availability. The database experiences heavy read activity. Every 4 hours, a full database export populates a staging database, causing unacceptable application latency and delays for the development team. A solutions architect needs a replacement architecture that resolves the latency and allows the development team to use the staging environment without delays. Which solution meets these requirements?

- ⊗ Use Amazon Aurora MySQL with Multi-AZ Aurora Replicas for production. Populate the staging database by implementing a backup and restore process that uses the mysqldump utility.

Using `mysqldump` will still cause application latency while the backup is being created, failing to meet the requirement of eliminating the latency issue during staging database creation.

- ✓ Use Amazon Aurora MySQL with Multi-AZ Aurora Replicas for production. Use database cloning to create the staging database on-demand.
- Aurora's database cloning feature creates a near-instantaneous copy of the database without impacting performance, allowing developers to use the staging environment immediately. Multi-AZ Aurora Replicas provide high availability.
- ✗ Use Amazon RDS for MySQL with a Multi-AZ deployment and read replicas for production. Use the standby instance for the staging database.
- The standby instance in a Multi-AZ RDS deployment is not intended for general read operations or staging environments. It's primarily for failover purposes and cannot be accessed directly for read operations.
- ✗ Use Amazon RDS for MySQL with a Multi-AZ deployment and read replicas for production. Populate the staging database by implementing a backup and restore process that uses the `mysqldump` utility.
- Similar to option A, using `mysqldump` will still cause application latency during the backup and restore process, failing to meet the requirements.

Overall Explanation:

The best solution is to use Amazon Aurora MySQL with Multi-AZ Aurora Replicas for the production environment and utilize Aurora's database cloning feature to create the staging database on-demand. Database cloning provides a fast and efficient way to create a fully functional staging environment without the latency issues associated with traditional backup and restore methods. The Multi-AZ Aurora Replicas ensure high availability for the production database.

[View Discussions](#)[Ask AI](#)

Question 34

Not Attempted

[Serverless](#)

A company hosts an employee web portal where employees can view payroll details. The company is implementing a new system that allows employees to upload scanned reimbursement documents. A program will extract text-based data from these documents and attach it to the corresponding employee's reimbursement ID for processing. The employee web portal requires 100% uptime. The document extraction program runs on demand throughout the day. The company requires a scalable and cost-effective solution with minimal changes to

the existing web portal and no code modifications. Which solution meets these requirements with the LEAST implementation effort?

- ☒ Run Amazon EC2 On-Demand Instances in an Auto Scaling group for the web portal. Use an AWS Lambda function to run the document extract program. Invoke the Lambda function when an employee uploads a new reimbursement document.
This solution utilizes Lambda, which is ideal for on-demand processing and scales automatically. Using On-Demand instances and Auto Scaling ensures 100% uptime for the web portal. It also requires no code changes.
- ☐ Run Amazon EC2 Spot Instances in an Auto Scaling group for the web portal. Run the document extract program on EC2 Spot Instances. Start document extract program instances when an employee uploads a new reimbursement document.
Spot Instances are not suitable for the web portal because they can be terminated by AWS at any time, violating the 100% uptime requirement.
- ☐ Purchase a Savings Plan to run the web portal and the document extract program. Run the web portal and the document extract program in an Auto Scaling group.
While a Savings Plan can reduce costs, this option doesn't specify how the document extraction program is run and doesn't guarantee minimal implementation effort or no code changes.
- ☐ Create an Amazon S3 bucket to host the web portal. Use Amazon API Gateway and an AWS Lambda function for the existing functionalities. Use the Lambda function to run the document extract program. Invoke the Lambda function when the API that is associated with a new document upload is called.
This option requires a significant architectural change to host the web portal on S3 and use API Gateway, which contradicts the requirement of minimal changes to the existing setup and potentially requires code changes.

Overall Explanation:

Option A provides the best balance between scalability, cost-effectiveness, minimal changes to the existing web portal, and adherence to the 100% uptime requirement. It leverages Lambda for on-demand processing and EC2 On-Demand instances with Auto Scaling for the web portal. Option B is incorrect because Spot Instances don't guarantee 100% uptime. Option C does not fully address the document extraction program. Option D requires major architectural changes that violate the requirement of minimal changes and no code modifications.

[View Discussions](#)[Ask AI](#)

Question 35

Not Attempted

Serverless



A media company has a multi-account AWS environment in the us-east-1 Region. The company has an Amazon Simple Notification Service (Amazon SNS) topic in a production account that publishes performance metrics. The company has an AWS Lambda function in an administrator account to process and analyze log data. The Lambda function in the administrator account must be invoked by messages from the SNS topic in the production account when significant metrics are reported. Which combination of steps will meet these requirements?

- ☒ Create an IAM resource policy for the Lambda function that allows Amazon SNS to invoke the function.

This allows the SNS topic to trigger the Lambda function. The resource policy grants SNS the necessary permissions to invoke the function, establishing cross-account access.

- ☐ Implement an Amazon Simple Queue Service (Amazon SQS) queue in the administrator account to buffer messages from the SNS topic in the production account. Configure the SQS queue to invoke the Lambda function.

While SQS could be used to buffer the messages, it introduces unnecessary complexity compared to directly invoking the lambda function using SNS.

- ☒ Create an IAM policy for the SNS topic that allows the Lambda function to subscribe to the topic.

This allows the Lambda function to subscribe to the SNS topic. The SNS topic access policy allows the Lambda function (in the administrator account) to subscribe to the topic in the production account, enabling cross-account communication.

- ☐ Use an Amazon EventBridge rule in the production account to capture the SNS topic notifications. Configure the EventBridge rule to forward notifications to the Lambda function that is in the administrator account.

This is an alternative, and valid, solution using EventBridge to manage and route the event notifications from the SNS topic to the Lambda function, providing a flexible event management system.

- ☐ Store performance metrics in an Amazon S3 bucket in the production account. Use Amazon Athena to analyze the metrics from the administrator account.

Storing metrics in S3 and using Athena is a valid solution for analyzing the data but does not address the requirement of invoking the Lambda function when significant metrics are reported in

real time.

Overall Explanation:

The combination of creating an IAM resource policy for the Lambda function and an IAM policy for the SNS topic correctly configures the cross-account access, allowing the SNS topic in the production account to invoke the Lambda function in the administrator account when significant metrics are reported. Option A grants SNS permission to invoke Lambda, and Option C grants Lambda permission to subscribe to the SNS topic.

[View Discussions](#)[Ask AI](#)[Question 36](#)[Not Attempted](#)[Networking](#)

A company is migrating an application from an on-premises location to Amazon Elastic Kubernetes Service (Amazon EKS). The company must use a custom subnet for pods that are in the company's VPC to comply with requirements. The company also needs to ensure that the pods can communicate securely within the pods' VPC. Which solution will meet these requirements?

- ☐ ☒ **Configure AWS Transit Gateway to directly manage custom subnet configurations for the pods in Amazon EKS.**
AWS Transit Gateway manages connectivity between multiple VPCs, on-premises networks, or other AWS services. It doesn't manage pod-level subnet configurations in Amazon EKS.
- ☐ ☒ **Create an AWS Direct Connect connection from the company's on-premises IP address ranges to the EKS pods.**
AWS Direct Connect establishes a dedicated network connection from on-premises to AWS. While useful for connectivity, it doesn't directly address the requirement of configuring custom subnets for pods within the VPC.
- ☒ ☐ **Use the Amazon VPC CNI plugin for Kubernetes. Define custom subnets in the VPC cluster for the pods to use.**
The Amazon VPC Container Network Interface (CNI) plugin is the default network plugin for Amazon EKS. It allows Kubernetes pods to receive IP addresses from a VPC's subnet and enables pods to communicate securely within the VPC as if they were native VPC resources. Defining custom subnets for the pods will fulfil the requirements.

- ⊗ Implement a Kubernetes network policy that has pod anti-affinity rules to restrict pod placement to specific nodes that are within custom subnets.

Pod anti-affinity rules are used for scheduling constraints and controlling which nodes pods can be scheduled onto. They don't handle subnet allocation or secure communication within the VPC.

Overall Explanation:

The Amazon VPC CNI plugin enables pods in Amazon EKS to obtain IP addresses from a subnet in your VPC, thereby enabling secure communication with other resources in the VPC. By defining custom subnets, you meet the compliance requirements. Therefore, using the Amazon VPC CNI plugin and defining custom subnets is the correct approach.

 View Discussions

 Ask AI

Question 37

Not Attempted

Database



A company hosts an ecommerce application that stores all data in a single Amazon RDS for MySQL DB instance that is fully managed by AWS. The company needs to mitigate the risk of a single point of failure with the LEAST implementation effort. Which solution meets these requirements?

- ✔ Modify the RDS DB instance to use a Multi-AZ deployment. Apply the changes during the next maintenance window.

This solution provides automated failover and high availability with minimal implementation effort. It leverages the built-in Multi-AZ feature of RDS.

- ⊗ Migrate the current database to a new Amazon DynamoDB Multi-AZ deployment. Use AWS Database Migration Service (AWS DMS) with a heterogeneous migration strategy to migrate the current RDS DB instance to DynamoDB tables.

Migrating to DynamoDB is a significant change that would require extensive application modifications and is more effort than necessary for simply achieving high availability. Also DynamoDB's Multi-AZ is for scaling and throughput, and not for the RDS-like automated failover that Multi-AZ RDS offers.

- ⊗ Create a new RDS DB instance in a Multi-AZ deployment. Manually restore the data from the existing RDS DB instance from the most recent snapshot.

Restoring from a snapshot is a manual process and does not provide automatic failover. Also requires setting up the new instance which is not the least effort.

- ⊗ Configure the DB instance in an Amazon EC2 Auto Scaling group with a minimum group size of three. Use Amazon Route 53 simple routing to distribute requests to all DB instances.

RDS is a managed service, it is not in EC2 and is incorrect to use an EC2 Auto Scaling group. This approach is also much more complex to implement and maintain than simply enabling Multi-AZ on the RDS instance.

Overall Explanation:

Modifying the RDS DB instance to use a Multi-AZ deployment is the solution that provides high availability with the least implementation effort. Multi-AZ deployments provide a standby instance in a different Availability Zone that automatically takes over if the primary instance fails. This offers a simple and efficient way to mitigate the risk of a single point of failure.

 View Discussions

 Ask AI

Question 38

Not Attempted

Storage



A company has multiple Microsoft Windows SMB file servers and Linux NFS file servers in an on-premises environment. As part of the company's AWS migration plan, the company wants to consolidate the file servers in the AWS Cloud. The company needs a managed AWS storage service that supports both NFS and SMB access and allows sharing between protocols. The solution must provide redundancy at the Availability Zone level. Which solution will meet these requirements?

- ✔ Use Amazon FSx for NetApp ONTAP for storage. Configure multi-protocol access.

Amazon FSx for NetApp ONTAP supports multi-protocol access (SMB, NFS) and offers Multi-AZ deployment for redundancy.

- ⊗ Create two Amazon EC2 instances. Use one EC2 instance for Windows SMB file server access and one EC2 instance for Linux NFS file server access.

This approach requires managing EC2 instances and configuring file sharing manually, which is not a managed solution.

- ⊗ Use Amazon FSx for NetApp ONTAP for SMB access. Use Amazon FSx for Lustre for NFS access.

This approach uses two different file systems which would not allow the files to be shared between protocols.

- ⊗ Use Amazon S3 storage. Access Amazon S3 through an Amazon S3 File Gateway.

Amazon S3 File Gateway does not directly support NFS or SMB protocols. It acts as a cache for S3 objects and is not a file system.

Overall Explanation:

Amazon FSx for NetApp ONTAP is a managed AWS storage service that supports both NFS and SMB access with multi-protocol access. It also supports Multi-AZ deployments, providing the required redundancy at the Availability Zone level.

 View Discussions

 Ask AI

Question 39

Not Attempted

Database



A software company is migrating a critical web application to AWS. The application currently runs on a single EC2 instance in a public subnet, hosting a MySQL database. DNS records are managed in Route 53. To enhance scalability, high availability, and reduce MySQL read latency, which combination of solutions should a solutions architect implement? (Choose two.)

- ⊗ Launch a second EC2 instance in a second AWS Region. Use a Route 53 failover routing policy to redirect traffic to the second EC2 instance.

This solution primarily addresses disaster recovery, not high availability within a region, and may not improve read latency.

- ✔ Create and configure an Auto Scaling group to launch private EC2 instances in multiple Availability Zones. Add the instances to a target group behind a new Application Load Balancer.

This solution enhances scalability and high availability by distributing the application across multiple Availability Zones within a region. The Application Load Balancer distributes traffic efficiently.

- ✔ Migrate the database to an Amazon Aurora MySQL cluster. Create the primary DB instance and reader DB instance in separate Availability Zones.

This solution improves read latency with the read replica and provides high availability by distributing the database across multiple Availability Zones.

- ⊗ Create and configure an Auto Scaling group to launch private EC2 instances in multiple AWS Regions. Add the instances to a target group behind a new Application Load Balancer.

While this provides disaster recovery capabilities, it doesn't directly address read latency and is more complex than needed for high availability.

- ⊗ Migrate the database to an Amazon Aurora MySQL cluster with cross-Region read replicas.

Cross-Region read replicas are useful for disaster recovery and global deployments, but creating a standard Aurora MySQL cluster with reader instances in separate AZs would be preferred and more efficient for decreasing the read latency.

Overall Explanation:

Options B and C together provide the best solution for scalability, high availability, and reduced read latency. Option B uses an Auto Scaling group and Application Load Balancer for scalable and highly available application instances across multiple Availability Zones within a single region. Option C utilizes Amazon Aurora MySQL with a primary DB instance and a reader DB instance in different Availability Zones to further improve read latency and provide database-level high availability. Options A, D, and E are not optimal because A and D are mainly for disaster recovery, and E is more than is needed when considering high availability.

 View Discussions

 Ask AI

Question 40

Not Attempted

Security



A company runs thousands of AWS Lambda functions. The company needs a solution to securely store sensitive information that all the Lambda functions use. The solution must also manage the automatic rotation of the sensitive information. Which combination of steps will meet these requirements with the LEAST operational overhead?

- ⊗ Create HTTP security headers by using Lambda@Edge to retrieve and create sensitive information

Lambda@Edge is primarily used for managing HTTP request/response behavior in a CDN environment with Amazon CloudFront. It is not designed for securely storing or rotating sensitive information.

- ✔ Create a Lambda layer that retrieves sensitive information
Creating a layer that securely retrieves sensitive information (for example, from AWS Secrets Manager), reduces duplication and operational overhead. The layer can also handle the logic for retrieving updated information if needed.
- ✔ Store sensitive information in AWS Secrets Manager
AWS Secrets Manager securely stores sensitive information and provides automatic rotation of secrets, reducing the need for manual management.
- ✘ Store sensitive information in AWS Systems Manager Parameter Store
AWS Systems Manager Parameter Store doesn't provide automatic rotation of secrets.
- ✘ Create a Lambda consumer with dedicated throughput to retrieve sensitive information and create environmental variables
Setting environment variables dynamically at runtime is not an efficient or scalable practice for 'thousands of Lambda functions'. Too much operational overhead.

Overall Explanation:

Storing sensitive information in AWS Secrets Manager allows for secure storage and automatic rotation. Using a Lambda layer allows multiple Lambda functions to access the information stored in Secrets Manager without needing to duplicate retrieval logic, therefore reduces operational overhead.

[View Discussions](#)[Ask AI](#)

Question 41

Not Attempted

[Compute](#)

A company has an internal application that runs on Amazon EC2 instances in an Auto Scaling group. The EC2 instances are compute optimized and use Amazon Elastic Block Store (Amazon EBS) volumes. The company wants to identify cost optimizations across the EC2 instances, the Auto Scaling group, and the EBS volumes. Which solution will meet these requirements with the MOST operational efficiency?

- ⊗ Create a new AWS Cost and Usage Report. Search the report for cost recommendations for the EC2 instances the Auto Scaling group, and the EBS volumes.

Creating a Cost and Usage Report can provide data for analysis but requires manual effort to identify specific recommendations, reducing operational efficiency.

- ⊗ Create new Amazon CloudWatch billing alerts. Check the alert statuses for cost recommendations for the EC2 instances, the Auto Scaling group, and the EBS volumes.

CloudWatch billing alerts provide notifications based on cost thresholds but do not offer specific recommendations for optimization.

- ✓ Configure AWS Compute Optimizer for cost recommendations for the EC2 instances, the Auto Scaling group and the EBS volumes.

AWS Compute Optimizer directly analyzes the utilization of your EC2 instances, Auto Scaling groups, and EBS volumes and provides specific, actionable recommendations for optimization, increasing operational efficiency.

- ⊗ Configure AWS Compute Optimizer for cost recommendations for the EC2 instances. Create a new AWS Cost and Usage Report. Search the report for cost recommendations for the Auto Scaling group and the EBS volumes.

While Compute Optimizer handles EC2 instances efficiently, relying on Cost and Usage Reports for Auto Scaling groups and EBS volumes introduces manual effort, reducing overall operational efficiency.

Overall Explanation:

AWS Compute Optimizer is the most operationally efficient solution because it automatically analyzes the resource utilization of EC2 instances, Auto Scaling groups, and EBS volumes to generate tailored cost optimization recommendations. Options A, B, and D require more manual effort and don't provide direct optimization guidance as efficiently as Compute Optimizer.

 View Discussions

 Ask AI

Question 42

Not Attempted

Storage



A company is running a media store across multiple Amazon EC2 instances distributed across multiple Availability Zones in a single VPC. The company wants a high-performing solution to

share data between all the EC2 instances, and prefers to keep the data within the VPC only. What should a solutions architect recommend?

⊗ Create an Amazon S3 bucket and call the service APIs from each instance's application
Using S3 would involve API calls for data access, which might not be the highest-performing solution for frequent data sharing. Also, it is not fully within the VPC.

⊗ Create an Amazon S3 bucket and configure all instances to access it as a mounted volume
S3 cannot be directly mounted as a volume to EC2 instances. This is not a viable solution.

⊗ Configure an Amazon Elastic Block Store (Amazon EBS) volume and mount it across all instances
EBS volumes can only be attached to a single EC2 instance at a time. They cannot be shared across multiple instances, especially in different Availability Zones.

✓ Configure an Amazon Elastic File System (Amazon EFS) file system and mount it across all instances
EFS is a network file system that can be mounted on multiple EC2 instances simultaneously. It provides high performance and allows instances in different Availability Zones within the same VPC to share data seamlessly. It fulfills the requirement of keeping data within the VPC.

Overall Explanation:

Amazon EFS is the best option because it allows multiple EC2 instances to share a file system simultaneously with high performance and is accessible across multiple Availability Zones within a VPC. S3 is not designed for file system semantics and EBS volumes can only be attached to a single instance.

 View Discussions

 Ask AI

Question 43

Not Attempted

Database



A company uses an Amazon RDS for MySQL instance. To prepare for end-of-year processing, the company added a read replica to accommodate extra read-only queries from the company's reporting tool. The read replica CPU usage was 60% and the primary instance CPU usage was 60%. After end-of-year activities are complete, the read replica has a constant 25% CPU usage. The primary instance still has a constant 60% CPU usage. The company wants to

rightsize the database infrastructure to optimize costs while maintaining sufficient performance for current operations and future growth. Which solution will meet these requirements?

☐ Delete the read replica. Do not make changes to the primary instance.

Deleting the read replica might impact future scalability and require re-configuration if read-heavy workloads increase again.

☒ Resize the read replica to a smaller instance size. Do not make changes to the primary instance.

Resizing the read replica to a smaller instance is a cost-effective solution since its CPU usage is now low. The primary instance does not need changes as it is operating at a manageable 60% CPU.

☐ Resize the read replica to a larger instance size. Resize the primary instance to a smaller instance size.

Resizing the read replica to a larger instance is unnecessary as the CPU usage is low. Decreasing the primary instance size could lead to performance issues given its constant 60% CPU utilization.

☐ Delete the read replica. Resize the primary instance to a larger instance.

Deleting the read replica eliminates read offloading, and increasing the primary instance size increases cost unnecessarily since read offloading is more cost effective.

Overall Explanation:

Since the read replica is now underutilized with only 25% CPU usage, it can be resized to a smaller instance to save costs while still handling the reduced read queries. No changes to the primary instance are needed, as it is consistently running at 60% CPU usage, which is manageable, and the read replica will still offload read queries in the future. Deleting the read replica risks future scalability issues and would require recreating and reconfiguring it in the future.

 View Discussions

 Ask AI

Question 44

Not Attempted

Serverless



A company is designing an application where users upload small files into Amazon S3. After a user uploads a file, the file requires a simple, one-time transformation to JSON format for later analysis. Each file must be processed as quickly as possible after it is uploaded. Demand will

vary significantly; some days see many uploads, others few or none. Which solution meets these requirements with the LEAST operational overhead?

- ⊗ Configure Amazon EMR to read text files from Amazon S3. Run processing scripts to transform the data. Store the resulting JSON file in an Amazon Aurora DB cluster.

Amazon EMR is an overkill for simple file processing. It's designed for large datasets and big data frameworks, adding unnecessary complexity and cost.

- ⊗ Configure Amazon S3 to send an event notification to an Amazon Simple Queue Service (Amazon SQS) queue. Use Amazon EC2 instances to read from the queue and process the data. Store the resulting JSON file in Amazon DynamoDB.

While S3 event notifications and SQS are good for decoupling, using EC2 instances introduces operational overhead for managing and scaling instances.

- ✓ Configure Amazon S3 to send an event notification to an Amazon Simple Queue Service (Amazon SQS) queue. Use an AWS Lambda function to read from the queue and process the data. Store the resulting JSON file in Amazon DynamoDB.

This is the most suitable option. It leverages serverless components (S3, SQS, Lambda, DynamoDB) for automated file processing without needing to manage servers. Lambda scales automatically based on demand, and DynamoDB is a suitable NoSQL database for storing JSON data.

- ⊗ Configure Amazon EventBridge (Amazon CloudWatch Events) to send an event to Amazon Kinesis Data Streams when a new file is uploaded. Use an AWS Lambda function to consume the event from the stream and process the data. Store the resulting JSON file in an Amazon Aurora DB cluster.

EventBridge and Kinesis Data Streams add unnecessary complexity for this use case. They are designed for real-time streaming of large data volumes. Aurora DB is also not ideal for storing JSON Data.

Overall Explanation:

The best solution is to use S3 event notifications to trigger an SQS queue, which in turn triggers an AWS Lambda function to process the file and store the result in DynamoDB. This is a serverless architecture that minimizes operational overhead and scales automatically with demand. EMR, EC2 instances, Kinesis, and Aurora DB add unnecessary complexity for this simple file transformation task.

 View Discussions

 Ask AI

Question 45

Not Attempted

Database



A company is migrating its databases to Amazon RDS for PostgreSQL. The company is also migrating its applications to Amazon EC2 instances. The company wants to optimize costs for long-running workloads and has a sufficient budget for upfront investments. Which cost optimization solution will meet this requirement MOST cost-effectively?

- ☐ Use On-Demand Instances for the Amazon RDS for PostgreSQL workloads. Purchase a 1 year Compute Savings Plan with the No Upfront option for the EC2 instances.

On-Demand instances and No Upfront Savings Plans are less cost-effective for long-running workloads.

- ☐ Purchase Reserved Instances for a 1 year term with the No Upfront option for the Amazon RDS for PostgreSQL workloads. Purchase a 1 year EC2 Instance Savings Plan with the No Upfront option for the EC2 instances.

No Upfront options provide the least savings. Shorter terms provide less savings than longer terms.

- ☐ Purchase Reserved Instances for a 1 year term with the Partial Upfront option for the Amazon RDS for PostgreSQL workloads. Purchase a 1 year EC2 Instance Savings Plan with the Partial Upfront option for the EC2 instances.

Partial Upfront and shorter terms provide less savings than All Upfront and longer terms, respectively.

- ☒ Purchase Reserved Instances for a 3 year term with the All Upfront option for the Amazon RDS for PostgreSQL workloads. Purchase a 3 year EC2 Instance Savings Plan with the All Upfront option for the EC2 instances.

This option offers the greatest cost savings for long-running workloads due to the 3-year term and All Upfront payment option. All Upfront offers the highest savings and 3 year term Reserved Instances/Savings Plan provide greater savings.

Overall Explanation:

The most cost-effective solution for long-running workloads, given sufficient budget for upfront payments, is to purchase Reserved Instances for a 3-year term with the All Upfront option for the Amazon RDS for PostgreSQL workloads and purchase a 3-year EC2 Instance Savings Plan with the All Upfront option for the EC2 instances. This maximizes cost savings through long-term commitment and upfront investment.

[View Discussions](#)[Ask AI](#)

Question 46

Not Attempted

Security



A company is using an Amazon Elastic Kubernetes Service (Amazon EKS) cluster. The company must ensure that Kubernetes service accounts in the EKS cluster have secure and granular access to specific AWS resources by using IAM roles for service accounts (IRSA). Which combination of solutions will meet these requirements?

- ☐ Create an IAM policy that defines the required permissions. Attach the policy directly to the IAM role of the EKS nodes.

Attaching the policy to the EKS node IAM role grants permissions to all pods running on the nodes, which does not provide granular access at the service account level and violates the principle of least privilege.
- ☐ Implement network policies within the EKS cluster to prevent Kubernetes service accounts from accessing specific AWS services.

Kubernetes network policies control network traffic between pods, but they do not manage access to AWS resources. AWS resource access is controlled by IAM.
- ☐ Modify the EKS cluster's IAM role to include permissions for each Kubernetes service account. Ensure a one-to-one mapping between IAM roles and Kubernetes roles.

Modifying the EKS cluster's IAM role directly is not the recommended approach for granting specific permissions to service accounts. It can become complex to manage and doesn't leverage the benefits of IRSA.
- ☒ Define an IAM role that includes the necessary permissions. Annotate the Kubernetes service accounts with the Amazon Resource Name (ARN) of the IAM role.

This option correctly describes part of the IRSA process. Annotating the service account with the IAM role ARN links the service account to the defined IAM role, granting the pods running under that service account the specified permissions.
- ☒ Set up a trust relationship between the IAM roles for the service accounts and an OpenID Connect (OIDC) identity provider.

This option is also a necessary part of the IRSA setup. The trust relationship allows the service account to assume the IAM role by presenting a signed JWT token issued by the OIDC provider.

Overall Explanation:

IAM roles for service accounts (IRSA) allows you to associate an IAM role with a Kubernetes service account. Pods that use the service account can then use the role to access AWS resources. This is achieved by creating an IAM role, defining a trust relationship with the OIDC provider associated with the EKS cluster, and annotating the Kubernetes service account with the IAM role's ARN. The trust relationship enables the Kubernetes service account to assume the IAM role securely. Therefore, defining the IAM role and setting up the OIDC trust relationship are both necessary steps to meet the requirements.

[View Discussions](#)[Ask AI](#)

Question 47

Not Attempted

Security



A company regularly uploads confidential data to Amazon S3 buckets for analysis. The company's security policies mandate that the objects must be encrypted at rest. The company must automatically rotate the encryption key every year. The company must be able to track key rotation by using AWS CloudTrail. The company also must minimize costs for the encryption key. Which solution will meet these requirements?

- ☐ Use server-side encryption with customer-provided keys (SSE-C)
SSE-C requires you to manage the keys, making automatic rotation and CloudTrail tracking more complex and potentially costly.
- ☐ Use server-side encryption with Amazon S3 managed keys (SSE-S3)
SSE-S3 doesn't allow tracking key rotation with CloudTrail and doesn't provide automatic key rotation yearly by the customer.
- ☒ Use server-side encryption with AWS KMS keys (SSE-KMS)
AWS-managed KMS keys are automatically rotated every year by AWS. Key rotation events are logged in CloudTrail, and there are no monthly fees associated with AWS managed keys beyond API call costs. Therefore, it is the least costly solution.
- ☐ Use server-side encryption with customer managed AWS KMS keys
Customer managed keys incur monthly fees and usage fees, making this a higher cost option. While providing more control, it's not the most cost-effective solution for this scenario where automatic rotation is desired and AWS-managed keys can fulfill requirements.

Overall Explanation:

SSE-KMS using AWS-managed keys is the best option. AWS automatically rotates these keys annually, which can be tracked via CloudTrail, and it minimizes costs because there is no additional cost beyond API calls. SSE-C requires manual management, and customer-managed KMS keys are more expensive than AWS managed keys. SSE-S3 doesn't allow key rotation tracking with CloudTrail.

[View Discussions](#)[Ask AI](#)

Question 48

Not Attempted

Monitoring and Logging



A company has migrated several applications to AWS in the past 3 months. The company wants to understand the breakdown of costs for each of these applications and receive a regular report containing this information. Which solution will meet these requirements MOST cost-effectively?

- ☐ Use AWS Budgets to download data for the past 3 months into a .csv file. Manually analyze the data to identify the desired information.
Downloading data from AWS Budgets and manually analyzing it is not a scalable or efficient solution for ongoing reporting.
- ☐ Load AWS Cost and Usage Reports into an Amazon RDS DB instance. Run SQL queries to extract the desired information.
While this solution works, setting up and managing an Amazon RDS instance solely for cost reporting is an overkill and less cost-effective than using Cost Explorer with tags.
- ☒ Tag all the AWS resources with a key for cost and a value of the application's name. Activate cost allocation tags. Use Cost Explorer to generate the reports.
This is the most cost-effective solution. Cost allocation tags and Cost Explorer allow for granular cost tracking and reporting without significant overhead.
- ☐ Tag all the AWS resources with a key for cost and a value of the application's name. Use the AWS Billing and Cost Management console to download bills for the past 3 months. Manually analyze the bills to determine costs.
Downloading and manually analyzing monthly bills is time-consuming and prone to errors, especially for a large number of applications and resources. Cost Explorer provides a more efficient and automated reporting solution.

availability compared to using a CDN.

- ⊗ Use AWS Lambda functions to serve the web application. Use Amazon Aurora Serverless v2 for the database. Store static assets in Amazon Elastic File System (Amazon EFS) One Zone-Infrequent Access (One Zone-IA).

While Lambda and Aurora Serverless v2 are scalable solutions, storing static assets in EFS One Zone-IA creates a single point of failure and is not suitable for high availability. Also, using Lambda for an entire web application might not be the most cost-effective solution, depending on the traffic patterns.

Overall Explanation:

Option B is the most resilient and highly available solution. Using an Auto Scaling group across multiple Availability Zones ensures that the application can withstand failures within a single Availability Zone. Multi-AZ RDS ensures database high availability. CloudFront, as a CDN, ensures availability and faster delivery of static assets by caching content at edge locations globally. This combination of services ensures continuous service and resilience in the face of failures.

 View Discussions

 Ask AI

Question 50

Not Attempted

Networking



An e-commerce company operates several internal applications across multiple AWS accounts managed using AWS Organizations. A security appliance residing in the company's networking account needs to inspect all network traffic between these applications, regardless of the AWS account origin. Which solution provides the MOST effective way to meet these requirements?

- ⊗ Deploy a Network Load Balancer (NLB) in the networking account. Configure the application accounts to route traffic to the NLB via an interface VPC endpoint.

NLBs are designed for TCP, UDP, and TLS traffic and are not the ideal choice for inspecting all application traffic, especially when a security appliance is involved. Interface VPC Endpoints work with NLBs, but do not provide the inspection capability directly.

- ⊗ Deploy an Application Load Balancer (ALB) in each application account and configure it to send traffic directly to the security appliance.

ALBs are designed for HTTP/HTTPS traffic and do not handle all types of application traffic.

Deploying ALBs in each application account also adds operational overhead and does not centralize inspection in the networking account.

- ✓ Deploy a Gateway Load Balancer (GWLB) in the networking account. Configure the application accounts to route traffic to the GWLB via a Gateway Load Balancer endpoint.

GWLB is specifically designed to route traffic to virtual appliances, such as security appliances, for inspection. Using a GWLB endpoint in each application account enables centralized inspection of all inter-application traffic within the networking account.

- ⊗ Deploy an interface VPC endpoint in each application account and configure it to send traffic directly to the security appliance.

Interface VPC endpoints provide private connectivity to AWS services, but they do not inherently provide a mechanism to route traffic through a security appliance for inspection.

Overall Explanation:

The correct answer is deploying a Gateway Load Balancer (GWLB) in the networking account and configuring the application accounts to route traffic to the GWLB via a Gateway Load Balancer endpoint. This solution is specifically designed for routing traffic to virtual appliances for inspection, fulfilling the stated requirements. GWLB centralizes the inspection process and simplifies the management of security policies across multiple AWS accounts.

 View Discussions

 Ask AI

Question 51

Not Attempted

Database



A company runs its production workload on an Amazon Aurora MySQL DB cluster that includes six Aurora Replicas. The company wants near-real-time reporting queries from one of its departments to be automatically distributed across three of the Aurora Replicas. Those three replicas have a different compute and memory specification from the rest of the DB cluster. Which solution meets these requirements?

- ✓ Create and use a custom endpoint for the workload

Correct. Custom endpoints allow you to group specific replicas and route traffic only to those replicas. You can include the three replicas with the specified compute and memory configurations, ensuring that reporting queries are distributed among these replicas.

⊗ Create a three-node cluster clone and use the reader endpoint

Incorrect. Creating a cluster clone is expensive, cumbersome, and unnecessary. The clone has separate storage and compute resources, adding complexity and cost.

⊗ Use any of the instance endpoints for the selected three nodes

Incorrect. While you can use the individual instance endpoints, this does not provide automatic distribution of the workload across the three nodes. It would require application-level logic to manage the connections and load balancing.

⊗ Use the reader endpoint to automatically distribute the read-only workload

Incorrect. The reader endpoint automatically distributes read queries across **ALL** Aurora Replicas in the cluster, not just the three with the specified configuration.

Overall Explanation:

The correct answer is to create and use a custom endpoint for the workload. Custom endpoints allow you to target a specific set of Aurora replicas for the reporting queries, enabling you to target the three replicas with different compute and memory specifications while distributing the workload across them. Cloning the cluster is costly and unnecessary. Reader endpoints distribute traffic across all replicas, not just the chosen three. Using individual instance endpoints does not provide automatic load distribution.

 View Discussions

 Ask AI

Question 52

Not Attempted

Serverless



A company runs a Node.js function on a server in its on-premises data center. The data center stores data in a PostgreSQL database. The company stores the database credentials in a connection string in an environment variable on the server. The company wants to migrate its application to AWS and replace the Node.js application server with AWS Lambda. The company also wants to migrate to Amazon RDS for PostgreSQL and ensure that the database credentials are securely managed. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Store the database credentials as a parameter in AWS Systems Manager Parameter Store. Configure Parameter Store to automatically rotate the secrets every 30 days. Update the Lambda function to retrieve the credentials from the parameter.
Parameter Store doesn't support automatic secret rotation, increasing operational overhead.
- ✔ Store the database credentials as a secret in AWS Secrets Manager. Configure Secrets Manager to automatically rotate the credentials every 30 days. Update the Lambda function to retrieve the credentials from the secret.
Secrets Manager is designed for managing secrets and supports automatic rotation, reducing operational overhead and improving security.
- ⊗ Store the database credentials as an encrypted Lambda environment variable. Write a custom Lambda function to rotate the credentials. Schedule the Lambda function to run every 30 days.
Writing and maintaining a custom rotation function increases operational overhead compared to using a managed service.
- ⊗ Store the database credentials as a key in AWS Key Management Service (AWS KMS). Configure automatic rotation for the key. Update the Lambda function to retrieve the credentials from the KMS key.
AWS KMS is for managing encryption keys, not secrets like database credentials, and doesn't provide direct support for managing or rotating database credentials.

Overall Explanation:

AWS Secrets Manager is specifically designed for storing and managing sensitive information like database credentials. It provides built-in functionality for securely storing, retrieving, and automatically rotating credentials. Secrets Manager can be configured to automatically rotate the database credentials at regular intervals (e.g., every 30 days). This reduces operational overhead by eliminating the need for manual credential rotation or custom rotation logic. Lambda functions can easily retrieve credentials stored in Secrets Manager by calling the Secrets Manager API, which simplifies the application code and enhances security. Other options are incorrect because they either don't support automatic rotation (Parameter Store), require custom rotation logic (Lambda environment variable), or are not designed for storing secrets (KMS).

[View Discussions](#)[Ask AI](#)

Question 53

Not Attempted

Database



A company is migrating an on-premises SQL Server database to Amazon RDS for SQL Server using AWS Database Migration Service (DMS). The database is very large, and the company wants to minimize downtime during the migration. The company needs a solution that allows them to keep the target RDS database up to date with changes made to the source database while the majority of the data is being initially loaded. Which of the following approaches will minimize downtime?

- ☐ Use AWS DMS to perform a full load migration, followed by a change data capture (CDC) task to replicate ongoing changes after the initial load is complete.

This option requires the source database to be unavailable during the full load, leading to significant downtime.

- ☐ Use AWS DMS to perform a full load migration and then use native SQL Server replication to replicate ongoing changes.

Using native SQL Server replication in conjunction with DMS requires more complex configuration and may not provide the same level of monitoring and management as DMS.

- ☒ Use AWS DMS to perform a full load and CDC simultaneously, replicating ongoing changes while the full load is in progress.

AWS DMS full load and CDC can be run simultaneously, minimizing downtime by replicating changes while the initial data is being loaded. This approach keeps the target database synchronized with the source database with minimal interruption to the application.

- ☐ Use AWS DMS to perform a homogeneous database migration with minimal settings. This will automatically replicate the data with zero downtime.

Homogeneous database migration does not automatically mean zero downtime, and minimal settings doesn't trigger a zero-downtime replication.

Overall Explanation:

The correct answer is to use AWS DMS to perform a full load and CDC simultaneously. This allows DMS to replicate ongoing changes while the initial data is loaded, minimizing downtime. This is the most efficient approach to keeping the target database up-to-date with the source database during migration.

[View Discussions](#)[Ask AI](#)

Question 54

Not Attempted

Networking



A company hosts a multi-tier web application with internal service components deployed on Amazon EC2 instances. These components require secure and private access to third-party Software as a Service (SaaS) APIs hosted on AWS, minimizing public internet exposure.

- ☐ Implement an AWS Site-to-Site VPN to establish a secure connection with the third-party SaaS provider.
A Site-to-Site VPN provides a secure connection but doesn't inherently minimize public internet exposure or guarantee private connectivity within the AWS environment.
- ☐ Deploy AWS Transit Gateway to manage and route traffic between the application's VPC and the third-party SaaS provider.
Transit Gateway is a regional virtual router that interconnects VPCs and on-premises networks. While it can manage traffic, it doesn't inherently provide the private connectivity required to minimize public internet exposure when connecting to SaaS APIs.
- ☐ Configure AWS PrivateLink to allow only outbound traffic from the VPC without enabling the third-party SaaS provider to establish connections back.
This option misrepresents PrivateLink's core benefit. PrivateLink provides bidirectional communication for secure access without public internet exposure. Limiting to outbound traffic hinders the SaaS provider's ability to fully establish sessions, potentially breaking the integration.
- ☒ Use AWS PrivateLink to create a private connection between the application's VPC and the third-party SaaS provider.
AWS PrivateLink allows you to privately connect your VPC to supported AWS services, services hosted by other AWS accounts (including SaaS providers), and supported AWS Marketplace partner services. It provides secure and private connectivity without exposing traffic to the public internet.

Overall Explanation:

AWS PrivateLink is the best solution because it establishes a private connection between the application's VPC and the third-party SaaS provider. This ensures secure and private connectivity, minimizes public internet exposure, and allows for bidirectional communication.

View Discussions

Ask AI

Question 55

Not Attempted

Database



An application allows users at a company's headquarters to access product data. The product data is stored in an Amazon RDS MySQL DB instance. The operations team has isolated an application performance slowdown and wants to separate read traffic from write traffic. A solutions architect needs to optimize the application's performance quickly. What should the solutions architect recommend?

- ☐ Change the existing database to a Multi-AZ deployment. Serve the read requests from the primary Availability Zone.

Multi-AZ deployments are primarily for high availability and disaster recovery. They do not separate read and write traffic.

- ☐ Change the existing database to a Multi-AZ deployment. Serve the read requests from the secondary Availability Zone.

In a Multi-AZ deployment, the secondary instance is a standby instance and cannot be used for read traffic.

- ☐ Create read replicas for the database. Configure the read replicas with half of the compute and storage resources as the source database.

Reducing the compute and storage resources of the read replicas may limit their ability to handle the read workload effectively, potentially negating the performance benefits of separating read traffic.

- ☒ Create read replicas for the database. Configure the read replicas with the same compute and storage resources as the source database.

Creating read replicas allows the application to offload read traffic from the source database, improving performance. Configuring the read replicas with the same compute and storage resources as the source database ensures they can handle the read workload efficiently.

Overall Explanation:

The optimal solution is to create read replicas with the same compute and storage resources as the source database. This configuration allows read traffic to be offloaded, improving the performance of the primary database instance, and ensures that the read replicas can handle the read workload without performance degradation. Multi-AZ is for disaster recovery, and using smaller read replicas could cause performance issues.

[View Discussions](#)[Ask AI](#)

Question 56

Not Attempted

Networking



A solutions architect needs to establish a secure connection between a company's on-premises corporate network and its Amazon VPC, enabling on-premises access to AWS resources. The solution must encrypt all traffic between the corporate network and the VPC at both the network and session layers. Additionally, it must implement security controls to prevent unrestricted access between AWS and the on-premises systems. Which solution satisfies these requirements?

- ☐ ☒ **Configure AWS Direct Connect to connect to the VPC. Configure the VPC route tables to allow and deny traffic between AWS and on premises as required.**
AWS Direct Connect provides a dedicated network connection, but it doesn't inherently provide encryption. It often needs to be used with VPN for encrypted traffic.
- ☐ ☒ **Create an IAM policy to allow access to the AWS Management Console only from a defined set of corporate IP addresses. Restrict user access based on job responsibility by using an IAM policy and roles.**
IAM policies are related to user access and permissions within AWS, not the network connection between on-premises and AWS.
- ☒ ☐ **Configure AWS Site-to-Site VPN to connect to the VPC. Configure route table entries to direct traffic from on premises to the VPC. Configure instance security groups and network ACLs to allow only required traffic from on premises.**
AWS Site-to-Site VPN uses IPsec (network layer) and SSL/TLS (session layer) to encrypt all traffic between the on-premises network and the AWS VPC. Route tables, security groups, and network ACLs are used to control traffic.
- ☐ ☒ **Configure AWS Transit Gateway to connect to the VPC. Configure route table entries to direct traffic from on premises to the VPC. Configure instance security groups and network ACLs to allow only required traffic from on premises.**
AWS Transit Gateway provides connectivity but might not by default ensure encryption at both the network and session layers. While security groups and ACLs provide access controls, encryption is a separate requirement.

Overall Explanation:

AWS Site-to-Site VPN is the best option because it uses IPsec, which encrypts traffic at the network layer and provides encryption at the session layer as well. Also the use of route tables, security groups, and NACLs allows granular control of the network traffic.

[View Discussions](#)[Ask AI](#)

Question 57

Not Attempted

Security



A company has a custom application with embedded credentials that retrieves information from a database in an Amazon RDS for MySQL DB cluster. The company needs to make the application more secure with minimal programming effort. The company has created credentials on the RDS for MySQL database for the application user. Which solution will meet these requirements?

- ☐ Store the credentials in AWS Key Management Service (AWS KMS). Create keys in AWS KMS. Configure the application to load the database credentials from AWS KMS. Enable automatic key rotation.

AWS KMS is used for encryption and key management, not for storing and rotating credentials themselves.

- ☐ Store the credentials in encrypted local storage. Configure the application to load the database credentials from the local storage. Set up a credentials rotation schedule by creating a cron job.

Manually setting up a cron job for credential rotation is less secure and more complex than using a managed service.

- ☒ Store the credentials in AWS Secrets Manager. Configure the application to load the database credentials from Secrets Manager. Set up a credentials rotation schedule by creating an AWS Lambda function for Secrets Manager.

AWS Secrets Manager securely stores and manages sensitive information such as credentials, and provides native support for automatic rotation using AWS Lambda, requiring minimal programming effort.

- ☐ Store the credentials in AWS Systems Manager Parameter Store. Configure the application to load the database credentials from Parameter Store. Set up a credentials rotation schedule in the RDS for MySQL database by using Parameter Store.

Parameter Store doesn't provide automatic rotation capabilities for database credentials.

Overall Explanation:

AWS Secrets Manager is designed to securely store, manage, and retrieve database credentials. It supports automatic credentials rotation for Amazon RDS databases with minimal programming effort by leveraging AWS Lambda. This approach offers a secure and integrated solution, reducing the burden of manual credential management.

[View Discussions](#)[Ask AI](#)

Question 58

Not Attempted

Serverless



A company wants to migrate its application to a serverless architecture. The serverless solution requires SQL-based analysis of both existing and newly generated data. The company currently stores the data in an Amazon S3 bucket. Data-at-rest encryption and cross-region replication for disaster recovery are mandatory requirements. Which solution satisfies these requirements while minimizing operational overhead?

- ☒ Create a new S3 bucket that uses server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Configure Cross-Region Replication (CRR). Load the data into the new S3 bucket. Use Amazon Athena to query the data.

Using SSE-KMS multi-Region keys provides robust encryption and CRR meets replication needs. Athena offers serverless SQL querying. However, creating a new bucket and migrating data adds operational overhead.

- ☐ Create a new S3 bucket that uses server-side encryption with Amazon S3 managed keys (SSE-S3). Configure Cross-Region Replication (CRR). Load the data into the new S3 bucket. Use Amazon RDS to query the data.

While SSE-S3 simplifies key management, Amazon RDS is not a serverless solution. Also SSE-S3 keys are region specific which makes them unsuitable for CRR. Creating a new bucket and migrating data also adds operational overhead.

- ☐ Configure Cross-Region Replication (CRR) on the existing S3 bucket. Use server-side encryption with Amazon S3 managed keys (SSE-S3). Use Amazon Athena to query the data.

Configuring CRR on the existing bucket minimizes operational overhead. Athena provides serverless SQL querying. SSE-S3 does not work well with cross region replication due to its region specific nature.

- ⊗ Configure S3 Cross-Region Replication (CRR) on the existing S3 bucket. Use server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Use Amazon RDS to query the data.

Configuring CRR on the existing bucket minimizes overhead. While SSE-KMS multi-Region keys offers better control, Amazon RDS is not a serverless SQL querying option.

Overall Explanation:

Option A is the most suitable answer. It uses AWS KMS multi-Region keys for encryption, which are compatible with Cross-Region Replication (CRR). It leverages Amazon Athena for querying the data, which is a serverless SQL query service. While it involves creating a new S3 bucket, this is necessary to enable server-side encryption with KMS multi-Region keys and utilize CRR effectively. While using the existing bucket may seem like a lower overhead approach, SSE-S3 is region specific and won't work with CRR. Option A is preferred over option D because it provides a serverless SQL option and over option B, because it uses a serverless SQL option and KMS multi-region keys.

 View Discussions

 Ask AI

Question 59

Not Attempted

Storage



A company has a web application with thousands of users. The application uses 8-10 user-uploaded images to generate AI images. Users can download the generated AI images once every 6 hours. The company also has a premium user option that allows users to download the generated AI images anytime. The company uses the user-uploaded images to run AI model training twice a year. The company needs a storage solution to store the images. Which storage solution meets these requirements MOST cost-effectively?

- ✔ Move uploaded images to Amazon S3 Glacier Deep Archive. Move premium user-generated AI images to S3 Standard. Move non-premium user-generated AI images to S3 Standard-Infrequent Access (S3 Standard-IA).

This option uses Glacier Deep Archive for infrequently accessed uploaded images, S3 Standard for premium users needing immediate access, and S3 Standard-IA for non-premium users with less frequent access, which is cost-effective.

- ⊗ Move uploaded images to Amazon S3 Glacier Deep Archive. Move all generated AI images to S3 Glacier Flexible Retrieval.

Glacier Flexible Retrieval would likely introduce unacceptable retrieval delays for premium users who require immediate access to download images. While it saves on cost, it does not fulfill the requirements.

- ⊗ Move uploaded images to Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA). Move premium user-generated AI images to S3 Standard. Move non-premium user-generated AI images to S3 Standard-Infrequent Access (S3 Standard-IA).

While S3 One Zone-IA is a lower-cost option, it provides less durability because it stores data in only one Availability Zone. It increases the risk of data loss for critical AI training data.

- ⊗ Move uploaded images to Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA). Move all generated AI images to S3 Glacier Flexible Retrieval.

S3 One Zone-IA is suitable for less critical data, but the retrieval delay from Glacier would negatively impact premium users.

Overall Explanation:

Option A provides the most cost-effective solution while meeting all requirements. S3 Glacier Deep Archive provides the lowest cost for long-term storage of the uploaded images needed for biannual training. S3 Standard provides immediate access for premium users. S3 Standard-IA provides cost savings for the non-premium user generated images that only need to be accessed once every 6 hours. S3 Glacier Flexible Retrieval is not a practical option due to retrieval delays.

[View Discussions](#)[Ask AI](#)

Question 60

Not Attempted

Serverless



A company develops machine learning (ML) models as independent microservices on AWS. Each microservice loads 1 GB of model data from Amazon S3 at startup. Users access the models through an asynchronous API, sending individual or batched requests. Usage patterns are irregular, with some models idle for extended periods and others experiencing bursts of thousands of requests. Which solution best meets these requirements?

- ⊗ Direct requests from the API to a Network Load Balancer (NLB). Deploy the ML models as AWS Lambda functions invoked by the NLB. Use auto scaling to scale the Lambda functions based on NLB traffic.

Lambda functions might face challenges with the 1 GB model data loading at startup due to cold starts and memory limitations. While auto-scaling is beneficial, loading the model on every invocation would be inefficient.

- ⊗ Direct requests from the API to an Application Load Balancer (ALB). Deploy the ML models as Amazon Elastic Container Service (Amazon ECS) services invoked by the ALB. Use auto scaling to scale the ECS cluster instances based on ALB traffic.

ALB with ECS provides good load balancing, but the irregular usage patterns might lead to idle ECS instances, increasing costs. The solution also lacks a mechanism to handle the asynchronous nature of the API effectively.

- ⊗ Direct requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the ML models as AWS Lambda functions that SQS events invoke. Use auto scaling to increase the number of vCPUs for the Lambda functions based on the size of the SQS queue.

Although SQS decouples the API from the ML processing and helps handling asynchronous requests, Lambda cold starts coupled with the 1GB model loading each time still present a challenge. Also, Lambda memory constraints might not be suitable for all model sizes.

- ✓ Direct requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the ML models as Amazon Elastic Container Service (Amazon ECS) services that read from the queue. Use auto scaling for Amazon ECS to scale both the cluster capacity and number of the services based on the size of the SQS queue.

This solution leverages SQS for asynchronous processing and ECS for containerized model execution. SQS decouples the API, handles bursts, and ensures reliable processing. ECS with auto scaling efficiently manages resources, scaling up or down based on demand while providing a persistent execution environment for the ML models to load the 1GB of data once.

Overall Explanation:

The optimal solution uses SQS to buffer asynchronous requests and ECS to host the ML models. SQS handles irregular traffic patterns and ensures that requests are reliably processed, while ECS provides a stable environment for the models to load the necessary data. Auto scaling for ECS based on the SQS queue size ensures efficient resource utilization and cost management. Thus option D is correct.

 View Discussions

 Ask AI

A company runs a web application on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB). The application stores data in an Amazon Aurora MySQL DB cluster. The company needs to create a disaster recovery (DR) solution. The acceptable recovery time for the DR solution is up to 30 minutes. The DR solution does not need to support customer usage when the primary infrastructure is healthy. Which solution will meet these requirements?

- ✔ Deploy the DR infrastructure in a second AWS Region with an ALB and an Auto Scaling group. Set the desired capacity and maximum capacity of the Auto Scaling group to a minimum value. Convert the Aurora MySQL DB cluster to an Aurora global database. Configure Amazon Route 53 for an active-passive failover with ALB endpoints.

This solution leverages Aurora Global Database for quick data replication and failover. The active-passive Route 53 configuration ensures that the DR environment is only used when the primary fails, and the Auto Scaling group's minimum capacity helps to minimize costs when idle.

- ✘ Deploy the DR infrastructure in a second AWS Region with an ALB. Update the Auto Scaling group to include EC2 instances from the second Region. Use Amazon Route 53 to configure active-active failover. Convert the Aurora MySQL DB cluster to an Aurora global database.

Active-active failover is not necessary because the DR solution should not support customer usage when the primary infrastructure is healthy.

- ✘ Back up the Aurora MySQL DB cluster data by using AWS Backup. Deploy the DR infrastructure in a second AWS Region with an ALB. Update the Auto Scaling group to include EC2 instances from the second Region. Use Amazon Route 53 to configure active-active failover. Create an Aurora MySQL DB cluster in the second Region. Restore the data from the backup.

Restoring a database from AWS Backup can exceed the 30-minute RTO requirement.

- ✘ Back up the infrastructure configuration by using AWS Backup. Use the backup to create the required infrastructure in a second AWS Region. Set the Auto Scaling group desired capacity to zero. Use Amazon Route 53 to configure active-passive failover. Convert the Aurora MySQL DB cluster to an Aurora global database.

Relying on restoring infrastructure configurations from backup will likely exceed the 30-minute RTO requirement. AWS Backup primarily backs up data, not infrastructure configuration.

Overall Explanation:

Option A is the best solution because it utilizes Aurora Global Database for fast replication and failover. It uses Route 53 active-passive failover to direct traffic to the DR environment only when needed and minimizes costs with a small Auto Scaling group footprint during normal operation. This approach meets the 30-minute RTO requirement and avoids unnecessary costs. Backups and restores will be too slow. Active-active is not required as per the question requirements.

[View Discussions](#)[Ask AI](#)

Question 62

Not Attempted

Storage



A company is migrating its data processing application to the AWS Cloud. The application processes several short-lived batch jobs that cannot be disrupted. Data is generated after each batch job is completed. The data must be accessed frequently for 30 days and retained for 2 years for compliance reasons. The company wants to minimize costs. Which of the following storage solutions is MOST cost-effective, while meeting the application requirements?

- ⊗ Migrate the data processing application to Amazon EC2 Spot Instances. Store the data in Amazon S3 Standard. Move the data to Amazon S3 Glacier Instant Retrieval after 30 days. Set an expiration to delete the data after 2 years.

Spot Instances can be disrupted, which violates the requirement that the batch jobs cannot be disrupted. S3 Glacier Instant Retrieval is more suited for long-term archival where frequent access is not required.

- ⊗ Migrate the data processing application to Amazon EC2 On-Demand Instances. Store the data in Amazon S3 Glacier Instant Retrieval. Move the data to S3 Glacier Deep Archive after 30 days. Set an expiration to delete the data after 2 years.

While On-Demand instances meet the non-disruptive requirement, using S3 Glacier Instant Retrieval for the initial 30 days when the data is accessed frequently is not cost-effective. S3 Glacier Instant Retrieval is designed for infrequently accessed data.

- ⊗ Deploy Amazon EC2 Spot Instances to run the batch jobs. Store the data in Amazon S3 Standard. Move the data to Amazon S3 Glacier Flexible Retrieval after 30 days. Set an expiration to delete the data after 2 years.

Spot Instances can be disrupted, which violates the requirement that the batch jobs cannot be disrupted.

- ✓ Deploy Amazon EC2 On-Demand Instances to run the batch jobs. Store the data in Amazon S3 Standard. Move the data to Amazon S3 Glacier Deep Archive after 30 days. Set an expiration to delete the data after 2 years.

On-Demand Instances prevent disruptions. S3 Standard provides cost-effective frequent access for 30 days. S3 Glacier Deep Archive is the cheapest option for long-term storage, meeting the 2-year retention policy. Setting an expiration prevents indefinite storage costs.

Overall Explanation:

The correct answer is D. On-Demand Instances ensure uninterrupted batch jobs. S3 Standard provides the most cost-effective solution for frequently accessed data (30 days). S3 Glacier Deep Archive is the most cost-effective for long-term storage (2 years) when data is infrequently accessed. The expiration policy manages data lifecycle and prevents unnecessary storage charges. Options A and C use Spot Instances, which are not suitable for jobs that cannot be disrupted. Option B uses S3 Glacier Instant Retrieval which is designed for infrequently accessed data that also needs to be retrieved quickly (milliseconds).

 View Discussions

 Ask AI

Question 63

Not Attempted

Networking



A company needs to design a hybrid network architecture. The company's workloads are currently stored in the AWS Cloud and in on-premises data centers. The workloads require single-digit millisecond latencies to communicate. The company uses an AWS Transit Gateway to connect multiple VPCs. Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

- ✗ Establish an AWS Site-to-Site VPN connection to each VPC.

A Site-to-Site VPN connection uses the public internet, which can introduce variable latency and is not the most cost-effective solution for low-latency requirements.

- ✓ Associate an AWS Direct Connect gateway with the transit gateway that is attached to the VPCs.

Associating a Direct Connect gateway with the transit gateway enables traffic to flow between the on-premises environment and the VPCs connected to the transit gateway. This offers dedicated, low-latency connectivity.

- ⊗ Establish an AWS Site-to-Site VPN connection to an AWS Direct Connect gateway.

This option is not necessary. Direct Connect already provides a dedicated connection. Adding a VPN on top of Direct Connect adds unnecessary complexity and doesn't significantly improve latency or cost.

- ✔ Establish an AWS Direct Connect connection. Create a transit virtual interface (VIF) to a Direct Connect gateway.

Establishing a Direct Connect connection and creating a transit VIF allows direct, low-latency access to the transit gateway and the connected VPCs. This is a cost-effective solution compared to multiple VPN connections.

- ⊗ Associate AWS Site-to-Site VPN connections with the transit gateway that is attached to the VPCs.

Using multiple Site-to-Site VPN connections is generally less cost-effective and provides higher latency than Direct Connect for connecting on-premises environments to a Transit Gateway.

Overall Explanation:

The most cost-effective way to achieve single-digit millisecond latencies between on-premises data centers and AWS workloads in this scenario is to use AWS Direct Connect. Associating a Direct Connect gateway with the Transit Gateway (Option B) and establishing a Direct Connect connection with a transit VIF (Option D) will establish a dedicated, low-latency connection between on-premises and AWS Cloud resources. VPN connections (Options A, C, and E) are less cost-effective and don't meet the latency requirements as effectively.

 View Discussions

 Ask AI

Question 64

Not Attempted

Database



A global ecommerce company runs its critical workloads on AWS, utilizing an Amazon RDS for PostgreSQL DB instance configured for a Multi-AZ deployment. Customers are experiencing application timeouts during database failovers, negatively impacting the user experience. The company requires a resilient solution to minimize failover time and eliminate these timeouts. Which of the following solutions would BEST meet these requirements?

- ✓ Create an Amazon RDS Proxy. Assign the proxy endpoint to the application to handle database connections.
RDS Proxy manages connections to the database efficiently. It reduces the impact of failovers by maintaining connections and transparently rerouting them to the standby instance during a failover event, thereby minimizing disruption and reducing application timeouts. This is the BEST option.
- ✗ Create a read replica for the DB instance. Redirect all read traffic to the read replica to reduce load on the primary instance.
While a read replica can improve read performance, it doesn't directly address failover times. Failovers will still occur on the primary instance, and the application will still experience timeouts during the process.
- ✗ Enable Performance Insights on the DB instance. Continuously monitor the CPU load and database performance to identify potential bottlenecks and performance issues.
Performance Insights helps identify performance issues, but it doesn't directly reduce failover time. It is a diagnostic tool, not a solution for minimizing failover disruption.
- ✗ Take regular automated snapshots of the DB instance. Copy the snapshots to multiple AWS Regions for disaster recovery purposes.
While snapshots are important for disaster recovery, they don't reduce the failover time during a Multi-AZ deployment. Restoring from a snapshot is a separate, longer process than a Multi-AZ failover.

Overall Explanation:

Amazon RDS Proxy is specifically designed to manage database connections and minimize the impact of failovers on applications. By maintaining persistent connections and transparently routing them to the standby instance during a failover, it significantly reduces failover time and eliminates application timeouts. Therefore, creating and utilizing an RDS Proxy is the most effective solution for meeting the company's requirements.

[View Discussions](#)[Ask AI](#)

Question 65

Not Attempted

Database



A company has multiple Amazon RDS DB instances that run in a development AWS account. All the instances have tags to identify them as development resources. The company needs the

development DB instances to run on a schedule only during business hours. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create an Amazon CloudWatch alarm to identify RDS instances that need to be stopped. Create an AWS Lambda function to start and stop the RDS instances.

CloudWatch alarms are designed to respond to metric thresholds rather than schedules, making this solution less suitable for scheduling start/stop actions.

- ⊗ Create an AWS Trusted Advisor report to identify RDS instances to be started and stopped. Create an AWS Lambda function to start and stop the RDS instances.

Trusted Advisor provides cost optimization and resource usage recommendations but cannot schedule start/stop actions.

- ⊗ Create AWS Systems Manager State Manager associations to start and stop the RDS instances.

Using AWS Systems Manager State Manager can be more complex for simple schedule-based start/stop actions compared to EventBridge and Lambda. However, it is a viable option and can be easier to implement than creating custom Lambda functions and EventBridge rules.

- ✔ Create an Amazon EventBridge rule that invokes AWS Lambda functions to start and stop the RDS instances.

This is the most straightforward solution. Amazon EventBridge allows you to create rules based on a schedule (using cron expressions) to automate tasks by triggering Lambda functions to handle the start/stop operations.

Overall Explanation:

Amazon EventBridge allows you to create rules based on a schedule (using cron expressions) to automate tasks. You can set up rules to start the RDS instances at the beginning of business hours and stop them at the end of business hours. By using AWS Lambda in conjunction with EventBridge, you can create functions that handle the start/stop operations automatically, leading to the least operational overhead.

 View Discussions

 Ask AI

[← Back to Exam](#)



Marketing

[Affiliate Program](#)

[Unlimited Access](#)

[Business Unlimited Access](#)

Support

[Contact Us](#)

team@itcertifications.io

Company

[Udemy Alternative](#)

[Whizlabs Alternative](#)

[TutorialsDojo Alternative](#)

Legal

[Terms of service](#)

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITCertifications Technologies. All rights reserved.